ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ

ΤΜΗΜΑ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ

“ΠΛΗΡΟΦΟΡΙΑΚΑ ΣΥΣΤΗΜΑΤΑ & ΥΠΗΡΕΣΙΕΣ”

ΚΑΤΕΥΘΥΝΣΗ: Προηγμένα Πληροφοριακά Συστήματα

ΜΑΘΗΜΑ: ΠΠΣ-181 - Ανάπτυξη Πληροφοριακών Συστημάτων και Διαχείριση Επιχειρησιακών Διεργασιών

Επιβλέποντες καθηγητές: *Βασιλακόπουλος Γεώργιος, Διαμαντοπούλου Βασιλική*

Στοιχεία Φοιτητών:

Ονοματεπώνυμο	***Αριθμός Μητρώου***
Σταύρος Ρισβάς	*Α.Μ. : 25083*
Πέτρος Σωτήριος Τζαβέλλας	*Α.Μ. : 25088*
Αγγελική Λαγογιάννη	*Α.Μ. : 25068*

Κύκλος - Ακαδημαϊκό Έτος: 2ος κύκλος σπουδών, Α’ εξάμηνο ακαδημαϊκού έτους 2025-2026

ΜΕΤΑΠΤΥΧΙΑΚΗ ΑΠΑΛΛΑΚΤΙΚΗ ΕΡΓΑΣΙΑ
2025-2026
ΜΕ ΤΙΤΛΟ:
«Ψηφιακός Μετασχηματισμός Ασφαλιστικών Υπηρεσιών: Ανάπτυξη Πληροφοριακού Συστήματος Self-Service μέσω Υπηρεσιοστρεφούς Αρχιτεκτονικής (SOA)»

Αθήνα
Φεβρουάριος 2026

Περιεχόμενα

1. Εισαγωγή 1
   1.1. Γενικό Πλαίσιο 1
   1.2. Περιγραφή της Προβληματικής Κατάστασης 1
   1.3. Σκοπός της Εργασίας 1
   1.4. Αντικείμενο και Λειτουργίες του Συστήματος 2
   1.5. Μεθοδολογία και Εργαλεία Ανάπτυξης 2
   1.6. Δομή της Εργασίας 3
2. Θεωρητικό Υπόβαθρο και Βιβλιογραφική Ανασκόπηση 3
   2.1. Ψηφιακός Μετασχηματισμός στην Ασφάλιση (InsurTech) 3
   2.2. Μεθοδολογία Εύπλαστων Συστημάτων (SSM) 3
   2.3. Ευέλικτη Ανάπτυξη Λογισμικού (Agile/Scrum) 4
   2.4. Υπηρεσιοστρεφής Αρχιτεκτονική (SOA) 4
   2.5. Η Προσέγγιση DevOps στην Ανάπτυξη ΠΣ 4
   2.6. Μοντέλα Αιτήματος-Εξυπηρέτησης (Request-Service Model) 4
   2.7. Ασφάλεια και Έλεγχος Προσβάσεων (RBAC) 5
3. Εφαρμογή της Μεθοδολογίας Μαλακών Συστημάτων (ΜΜΣ) 5
   3.1. Περιγραφή της Προβληματικής Κατάστασης (Unstructured Problem Situation) 5
   3.2. Η Πλούσια Εικόνα (Rich Picture) 5
   3.3. Ανάλυση CATWOE 6
   3.4. Ορισμός Ρίζας (Root Definition) 7
   3.5. Εννοιολογικό Μοντέλο (Conceptual Model) 7
   3.6. Σύγκριση Εννοιολογικού Μοντέλου με την Υφιστάμενη Κατάσταση 8
4. Διαδικασία Ανάπτυξης Συστήματος και Ανάλυση 9
   4.1. Επιλογή και Αιτιολόγηση Μεθοδολογίας Agile (SCRUM) 9
   4.1.1. Συγκριτική Ανάλυση: Waterfall vs Agile-SCRUM 9
   4.1.2. Σύγκριση με άλλες Agile Μεθοδολογίες (Kanban) 10
   4.1.3. Ρόλοι Scrum 10
   4.2. Φάσεις Διαδικασίας Ανάπτυξης 10
   4.3. Product Backlog (Λίστα Απαιτήσεων) 11
   4.3.1. Προτεραιοποίηση Απαιτήσεων (MoSCoW Method) 11
   4.3.2. Καταγραφή Ιστοριών Χρήστη (User Stories). 11
   4.4. Προγραμματισμός Sprints και Story Points 12
   4.5. Μοντελοποίηση Συστήματος 13

4.5.1. Μοντελοποίηση Επιχειρησιακών Διεργασιών (BPMN) 13
4.5.2. Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram) 15
4.5.3. Διάγραμμα Κλάσεων (Class Diagram) 16
4.5.4. Διάγραμμα Αλληλουχίας (Sequence Diagram) 18
4.5.5. Διάγραμμα Δραστηριοτήτων 19
4.5.5.1. Τροποποίηση Συμβολαίου και Καλύψεων 20
4.5.5.2. Διαδικασία Ηλεκτρονικής Πληρωμής 21
4.6. Απαιτήσεις Ασφάλειας και Ελέγχου Πρόσβασης (Security & Authorizations) 21
4.7. Αρχιτεκτονική Ασφάλειας και Προστασίας Δεδομένων 22
5. Πραγμάτωση Συστήματος 23
5.1. Περιβάλλον Ανάπτυξης και Εργαλεία 23
5.2. Υλοποίηση Βάσης Δεδομένων (Database Schema) 24
5.3. Υλοποίηση Λογικής Συστήματος (Backend Logic - Java) 25
5.3.1. Κλάση Υπηρεσίας (Service Layer) 25
5.3.2. Κλάση Ελέγχου (REST Controller) 26
5.3.3. Υλοποίηση Ασφάλειας και Εξουσιοδότησης (Spring Security) 27
5.4. Ενορχήστρωση Επιχειρησιακών Διεργασιών με Bonita BPM 31
5.5. Αυτοματοποίηση και Containerization (DevOps Approach) 32
5.6. Διεπαφή Χρήστη (User Interface) 33
5.6.1. Οθόνη Ταυτοποίησης (Login Screen) 33
5.6.2. Πίνακας Ελέγχου (Dashboard) 34
5.6.3. Διαχείριση και Τροποποίηση Καλύψεων (Underwriting) 34
5.6.4. Οικονομική Διαχείριση και Πληρωμές 35
5.6.5. Επεξεργασία Στοιχείων Πελάτη 35
5.7. Τεκμηρίωση Διεπαφών (API Documentation) 36
5.8. Πίνακας Ιχνηλασιμότητας Απαιτήσεων (Traceability Matrix) 36
6. Συμπεράσματα και Μελλοντικές Επεκτάσεις 37
6.1. Συμπεράσματα 37
6.2. Μελλοντικές Επεκτάσεις 37
7. Βιβλιογραφία 38

Ευρετήριο Πινάκων

Πίνακας 1: Σύγκριση Εννοιολογικού Μοντέλου με την Υφιστάμενη Κατάσταση 9
Πίνακας 2: Σύγκριση Μεθοδολογιών Agile και Waterfall 10
Πίνακας 3: Καταγραφή User Stories 12
Πίνακας 4: Παρουσίαση Επαναληπτικών Κύκλων Ανάπτυξης 12
Πίνακας 5: Σενάριο Χρήσης- Ηλεκτρονική Πληρωμή 16
Πίνακας 6: Μήτρα Εξουσιοδοτήσεων Χρηστών 22
Πίνακας 7: Πίνακας Ιχνηλασιμότητας Απαιτήσεων 36

Ευρετήριο Εικόνων

Εικόνα 1: Πλούσια Εικόνα της υφιστάμενης προβληματικής κατάστασης στην Ασφαλιστική Εταιρεία.....6
Εικόνα 2: Εννοιολογικό μοντέλο.....8
Εικόνα 3: Μοντελοποίηση της διεργασίας αυτοεξυπηρέτησης σε περιβάλλον SOA με χρήση διακριτών ζωνών ευθύνης (Swimlanes).....14
Εικόνα 4: Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram) του Web Portal Ασφαλιστικής.....15
Εικόνα 5: Εννοιολογικό μοντέλο (Conceptual Model) των βασικών οντοτήτων του συστήματος.....17
Εικόνα 6: Αναλυτικό Διάγραμμα Κλάσεων (UML Class Diagram) με τύπους δεδομένων και λειτουργικές μεθόδους.....18
Εικόνα 7: Διάγραμμα Αλληλουχίας (Sequence Diagram) για τη δυναμική διαδικασία υπολογισμού νέου ασφαλίστρου (Underwriting).....19
Εικόνα 8: Διάγραμμα Δραστηριοτήτων για τη διαδικασία τροποποίησης καλύψεων συμβολαίου. 20
Εικόνα 9: Διάγραμμα Δραστηριοτήτων για την ολοκλήρωση ηλεκτρονικής πληρωμής. 21
Εικόνα 10: Μοντελοποίηση της επιχειρησιακής διαδικασίας «Insurance Process» σε περιβάλλον Bonita BPM Studio. 32
Εικόνα 11: Παραμετροποίηση του REST POST Connector για την κλήση του εξωτερικού API υπολογισμού ασφαλίστρου. 32
Εικόνα 12: Οθόνη εισόδου στο MyInsurance Portal 33
Εικόνα 13: Κεντρικός Πίνακας Ελέγχου χρήστη.....34
Εικόνα 14: Δυναμική τροποποίηση καλύψεων και υπολογισμός κόστους.....34
Εικόνα 15: Οθόνη οικονομικής διαχείρισης και ιστορικού πληρωμών.....35
Εικόνα 16: Φόρμα επεξεργασίας προσωπικών στοιχείων πελάτη. 35

1. Εισαγωγή

1.1. Γενικό Πλαίσιο

Ο κλάδος της ιδιωτικής ασφάλισης στην Ελλάδα διέρχεται μια περίοδο έντονου ψηφιακού μετασχηματισμού. Οι σύγχρονες καταναλωτικές συνήθειες, επηρεασμένες από την ταχύτητα του διαδικτύου και την ευκολία των ηλεκτρονικών συναλλαγών, δημιουργούν νέες απαιτήσεις για τις ασφαλιστικές εταιρείες. Η παραδοσιακή προσέγγιση εξυπηρέτησης, η οποία βασίζεται σε φυσική παρουσία ή τηλεφωνική επικοινωνία, τείνει να αντικατασταθεί από ψηφιακά κανάλια που προσφέρουν αμεσότητα, διαφάνεια και ευελιξία. Σε αυτό το πλαίσιο, η ανάπτυξη διαδικτυακών πυλών (Web Portals) για την αυτοεξυπηρέτηση των πελατών αποτελεί πλέον στρατηγική προτεραιότητα για τη βιωσιμότητα και την ανταγωνιστικότητα των ασφαλιστικών οργανισμών.

Αυτά τα συστήματα δεν είναι απλά εργαλεία επικοινωνίας, αλλά ολοκληρωμένες πλατφόρμες που ενοποιούν τα δεδομένα των πελατών, παρέχουν υπηρεσίες σε πραγματικό χρόνο και επιτρέπουν τη διασύνδεση με τρίτα συστήματα (π.χ. τράπεζες για πληρωμές) μέσω υπηρεσιοστρεφούς αρχιτεκτονικής (SOA).

1.2. Περιγραφή της Προβληματικής Κατάστασης

Στην παρούσα φάση, η υπό μελέτη ασφαλιστική εταιρεία αντιμετωπίζει σημαντικές λειτουργικές δυσχέρειες που πηγάζουν από την έλλειψη ενός ολοκληρωμένου συστήματος ηλεκτρονικής εξυπηρέτησης. Οι πελάτες, προκειμένου να ενημερωθούν για τις παροχές των συμβολαίων τους ή να αιτηθούν απλές αλλαγές (όπως τροποποίηση προσωπικών στοιχείων ή αλλαγή τρόπου πληρωμής), υποχρεώνονται να επικοινωνούν τηλεφωνικά με το κέντρο εξυπηρέτησης ή να αποστέλλουν αιτήματα μέσω ηλεκτρονικού ταχυδρομείου. Η διαδικασία αυτή δημιουργεί μια προβληματική κατάσταση (problem situation), η οποία χαρακτηρίζεται από:

- Μεγάλους χρόνους αναμονής για τους πελάτες.
- Αυξημένο διοικητικό φόρτο για τους υπαλλήλους της εταιρείας.
- Κίνδυνο λαθών κατά τη χειροκίνητη καταχώρηση δεδομένων.
- Αδυναμία άμεσης πληρωμής και ανανέωσης συμβολαίων εκτός εργάσιμων ωρών.
- Έλλειψη διαφάνειας, καθώς ο πελάτης δεν έχει άμεση εικόνα της εξέλιξης των αιτημάτων του σε πραγματικό χρόνο.

1.3. Σκοπός της Εργασίας

Σκοπός της παρούσας εργασίας είναι η ανάλυση, ο σχεδιασμός και η ανάπτυξη ενός σύγχρονου Πληροφοριακού Συστήματος (Web Portal), το οποίο θα επιτρέπει την πλήρη ψηφιακή διαχείριση του ασφαλιστικού χαρτοφυλακίου από τον ίδιο τον πελάτη (Self-Service). Το σύστημα στοχεύει στην αυτοματοποίηση των διαδικασιών, τη μείωση του λειτουργικού κόστους και τη βελτίωση της εμπειρίας του χρήστη, παρέχοντας ασφαλή και ελεγχόμενη πρόσβαση σε ευαίσθητα προσωπικά και οικονομικά δεδομένα.

1.4. Αντικείμενο και Λειτουργίες του Συστήματος

Το προτεινόμενο σύστημα θα καλύπτει ένα ευρύ φάσμα λειτουργιών, σχεδιασμένων για να καλύψουν τις ανάγκες του σύγχρονου ασφαλισμένου. Συγκεκριμένα, οι βασικές λειτουργικές απαιτήσεις του συστήματος περιλαμβάνουν:

1. Συγκεντρωτική Προβολή Χαρτοφυλακίου: Οι χρήστες θα έχουν τη δυνατότητα να βλέπουν αναλυτικά τα συμβόλαιά τους, κατηγοριοποιημένα ανά κλάδο ασφάλισης:
    - Αυτοκινήτου
    - Υγείας
    - Ζωής
    - Κατοικίας

Κάθε κατηγορία θα παρέχει πληροφορίες για τις τρέχουσες παροχές, τα όρια κάλυψης και τις ημερομηνίες ανανέωσης.

2. Διαχείριση Προσωπικών Στοιχείων: Παροχή δυνατότητας στον χρήστη για άμεση επικαιροποίηση των στοιχείων του, όπως:
    - Στοιχεία επικοινωνίας και κατοικίας (Διεύθυνση).
    - Επαγγελματικά στοιχεία.
    - Στοιχεία ταυτοπροσωπίας (Αριθμός Δελτίου Ταυτότητας).
3. Οικονομική Διαχείριση και Πληρωμές:
    - Ηλεκτρονική Πληρωμή: Δυνατότητα άμεσης εξόφλησης ασφαλίστρων και ανανεώσεων μέσω του Portal.
    - Τρόπος Πληρωμής: Επιλογή για ενεργοποίηση ή απενεργοποίηση πάγιας εντολής πιστωτικής κάρτας.
    - Συχνότητα Πληρωμής: Δυνατότητα αλλαγής της συχνότητας καταβολής ασφαλίστρων (Μηνιαία, Τριμηνιαία, Εξαμηνιαία ή Ετήσια) ανάλογα με τις ανάγκες του πελάτη.
4. Δυναμική Τροποποίηση Συμβολαίου (Underwriting): Οι χρήστες θα μπορούν να προσαρμόζουν το συμβόλαιό τους προσθέτοντας ή αφαιρώντας καλύψεις, με το σύστημα να υπολογίζει και να εμφανίζει αυτόματα το νέο ασφάλιστρο προς έγκριση.

1.5. Μεθοδολογία και Εργαλεία Ανάπτυξης

Για την προσέγγιση της προβληματικής κατάστασης και τον ορισμό των απαιτήσεων θα χρησιμοποιηθεί η Μεθοδολογία Εύπλαστων Συστημάτων (Soft Systems Methodology - SSM). Η επιλογή αυτή αιτιολογείται από το γεγονός ότι το πεδίο μελέτης περιλαμβάνει έντονο τον ανθρώπινο παράγοντα και οργανωτικές πολυπλοκότητες που απαιτούν ποιοτική ανάλυση πριν την τεχνική υλοποίηση. Η ανάπτυξη του λογισμικού θα ακολουθήσει την Agile μεθοδολογία (SCRUM), επιτρέποντας την σταδιακή και επαναληπτική υλοποίηση των λειτουργιών. Η διαδικασία ανάπτυξης θα οργανωθεί με βάση τις τρεις θεμελιώδεις φάσεις ενός έργου Πληροφοριακών Συστημάτων:

1. Φάση Σύλληψης (Conception Phase): Προσδιορισμός οράματος και αναγκών.
2. Φάση Ορισμού (Definition Phase): Μοντελοποίηση και προδιαγραφή απαιτήσεων.
3. Φάση Εκτέλεσης (Execution Phase): Υλοποίηση, έλεγχος και εγκατάσταση.

Τεχνικά, το σύστημα θα υιοθετήσει μια Υπηρεσιοστρεφή Αρχιτεκτονική (Service-Oriented Architecture - SOA), όπου κάθε επιχειρησιακή λειτουργία (π.χ. πληρωμή, έκδοση συμβολαίου) θα υλοποιείται ως διακριτή υπηρεσία. Για την πραγμάτωση θα χρησιμοποιηθεί αντικειμενοστραφής γλώσσα προγραμματισμού (Java) και σχεσιακή βάση δεδομένων για την αποθήκευση των πληροφοριών.

1.6. Δομή της Εργασίας

Η εργασία δομείται ως εξής:

- Κεφάλαιο 2: Περιλαμβάνει τη βιβλιογραφική ανασκόπηση σχετικά με τις μεθοδολογίες ανάπτυξης ΠΣ και την SSM.
- Κεφάλαιο 3: Παρουσιάζει την εφαρμογή της Μεθοδολογίας Εύπλαστων Συστημάτων (SSM), συμπεριλαμβανομένης της Πλούσιας Εικόνας (Rich Picture) και του ορισμού CATWOE.
- Κεφάλαιο 4: Περιγράφει τη διαδικασία ανάπτυξης (Agile/Scrum) και την ανάλυση του συστήματος μέσω διαγραμμάτων UML (Use Case, Class, Activity Diagrams).
- Κεφάλαιο 5: Αφορά την τεχνική υλοποίηση (Πραγμάτωση) του συστήματος, με παρουσίαση του κώδικα και του γραφικού περιβάλλοντος.

2. Θεωρητικό Υπόβαθρο και Βιβλιογραφική Ανασκόπηση

2.1. Ψηφιακός Μετασχηματισμός στην Ασφάλιση (InsurTech)

Η σύγχρονη βιβλιογραφία αναγνωρίζει τον ψηφιακό μετασχηματισμό ως κρίσιμο παράγοντα επιβίωσης για τους ασφαλιστικούς οργανισμούς. Οι τεχνολογίες διαδικτύου επιτρέπουν τη μετάβαση από το παραδοσιακό μοντέλο πωλήσεων ("push" model) σε μοντέλα εστιασμένα στον πελάτη ("pull" model), όπου οι χρήστες αναζητούν και διαχειρίζονται υπηρεσίες αυτόνομα. Η αρχιτεκτονική του προτεινόμενου συστήματος εναρμονίζεται με τις αρχές της Κυβερνητικής (Cybernetics), καθώς λειτουργεί ως ένα αυτορυθμιζόμενο σύστημα που χρησιμοποιεί βρόχους ανάδρασης (feedback loops) για τη βελτιστοποίηση των ασφαλιστικών διεργασιών. Η προσέγγιση αυτή ακολουθεί το μοντέλο των αυτο-οργανωμένων συστημάτων αναφορικά με τη διαχείριση της πληροφορίας (Kybernetes, 2003). Για παράδειγμα στο περιγραφόμενο Web Portal, ένας βρόχος ανάδρασης δημιουργείται όταν ο πελάτης αλλάζει τη συχνότητα πληρωμής. Το σύστημα ενημερώνει αυτόματα το ποσό του ασφαλίστρου και επιστρέφει την πληροφορία στον πελάτη για έγκριση. Αυτή η αυτορύθμιση μειώνει την ανάγκη για παρέμβαση υπαλλήλου, ακολουθώντας την κυβερνητική αρχή της μείωσης της πολυπλοκότητας.

2.2. Μεθοδολογία Εύπλαστων Συστημάτων (SSM)

Η SSM (Checkland, 1981) είναι μια ολιστική προσέγγιση για την αντιμετώπιση προβληματικών καταστάσεων όπου εμπλέκονται ανθρώπινοι παράγοντες και διαφορετικές κοσμοθεωρίες. Σε αντίθεση με τις «σκληρές» μεθοδολογίες που εστιάζουν στο πώς θα υλοποιηθεί μια λύση, η SSM εστιάζει στο τι πρέπει να γίνει, χρησιμοποιώντας εργαλεία όπως οι Πλούσιες Εικόνες και η ανάλυση CATWOE για την κατανόηση του προβλήματος πριν την τεχνική υλοποίηση. Στην ασφαλιστική, η SSM επιτρέπει να δούμε ότι το πρόβλημα δεν είναι μόνο η έλλειψη λογισμικού, αλλά και

η δυσαρέσκεια των πελατών από την αναμονή στο τηλέφωνο. Η «κοσμοθεωρία» (Weltanschauung) του πελάτη είναι η άμεση εξυπηρέτηση, ενώ της εταιρείας είναι η μείωση του λειτουργικού κόστους. Το ΠΣ πρέπει να ικανοποιεί και τις δύο αυτές οπτικές.

2.3. Ευέλικτη Ανάπτυξη Λογισμικού (Agile/Scrum)

Η ανάπτυξη λογισμικού έχει μετακινηθεί από τα γραμμικά μοντέλα όπως το μοντέλο του Καταράκτη (Waterfall) σε ευέλικτες (Agile) προσεγγίσεις. Η Agile ανάπτυξη δεν είναι μια συγκεκριμένη μεθοδολογία, αλλά μια ευρύτερη φιλοσοφία που βασίζεται στις αρχές του «Μανιφέστου για την Ευέλικτη Ανάπτυξη Λογισμικού». Δίνει έμφαση στην προσαρμοστικότητα, την άμεση επικοινωνία μεταξύ των μελών της ομάδας και την παράδοση λειτουργικού λογισμικού σε μικρά, τακτικά χρονικά διαστήματα. Ενώ η Agile ορίζει τις αξίες και τις αρχές, το SCRUM (Schwaber & Sutherland, 2020) αποτελεί το δημοφιλέστερο πλαίσιο για την υλοποίησή τους, καθώς επιτρέπει τη διαχείριση αβεβαιότητας και αλλαγών μέσω επαναληπτικών κύκλων (Sprints). Σύμφωνα με τη βιβλιογραφία, ο συνδυασμός SSM (για την ανάλυση απαιτήσεων) και Agile (για την ανάπτυξη) προσφέρει τα βέλτιστα αποτελέσματα σε σύνθετα επιχειρησιακά περιβάλλοντα. Για παράδειγμα αντί να δημιουργηθεί όλο το Portal μαζί, με το Scrum θα αναπτυχθεί στο Sprint 1 η λειτουργία «Προβολή Συμβολαίου». Μόλις ο πελάτης δώσει feedback ότι η εμφάνιση είναι σωστή, ακολουθεί το Sprint 2 για την «Ηλεκτρονική Πληρωμή». Έτσι, μειώνεται ο κίνδυνος να δημιουργηθεί κάτι που δεν λειτουργεί.

2.4. Υπηρεσιοστρεφής Αρχιτεκτονική (SOA)

Για την υλοποίηση του συστήματος, κρίνεται αναγκαία η χρήση της Υπηρεσιοστρεφούς Αρχιτεκτονικής (Service-Oriented Architecture - SOA). Η SOA επιτρέπει στο ΠΣ να λειτουργεί ως ένα σύνολο αυτόνομων υπηρεσιών που επικοινωνούν μεταξύ τους. Για παράδειγμα η λειτουργία «Πληρωμή Ασφαλίστρων» υλοποιείται ως μια ανεξάρτητη υπηρεσία. Αν στο μέλλον η εταιρεία αλλάξει τράπεζα συνεργασίας, θα χρειαστεί να τροποποιηθεί μόνο η υπηρεσία πληρωμής, χωρίς να επηρεαστεί το υπόλοιπο Portal (π.χ. η προβολή των στοιχείων υγείας).

2.5. Η Προσέγγιση DevOps στην Ανάπτυξη ΠΣ

Η σύγχρονη βιβλιογραφία προκρίνει τη συνδυαστική χρήση της Agile Methodology με την κουλτούρα DevOps (Development & Operations). Ενώ η Agile εστιάζει στην ευελιξία των αλλαγών, το DevOps στοχεύει στην αυτοματοποίηση της μεταφοράς του κώδικα στο περιβάλλον λειτουργίας και στη συνεχή παρακολούθηση του συστήματος. Για το Portal της ασφαλιστικής, η προσέγγιση DevOps διασφαλίζει ότι κάθε νέα βελτίωση (π.χ. μια νέα μέθοδος πληρωμής) θα ελέγχεται αυτόματα και θα ενσωματώνεται στο σύστημα χωρίς να διακόπτεται η λειτουργία του για τους υπόλοιπους πελάτες.

2.6. Μοντέλα Αιτήματος-Εξυπηρέτησης (Request-Service Model)

Οι επιχειρησιακές διεργασίες στον κλάδο της ασφάλισης ακολουθούν το πρότυπο Request-Service (ή Client-Server). Σε αυτό το μοντέλο, ένας ρόλος (π.χ. ο Πελάτης)

εκκινεί ένα αίτημα και ένας άλλος ρόλος ή υπηρεσία (π.χ. το σύστημα Underwriting ή ένας υπάλληλος) ανταποκρίνεται παρέχοντας την αντίστοιχη υπηρεσία.

2.7. Ασφάλεια και Έλεγχος Προσβάσεων (RBAC)

Η διαχείριση ευαίσθητων δεδομένων απαιτεί την εφαρμογή μοντέλων Ελέγχου Πρόσβασης βάσει Ρόλων (Role-Based Access Control - RBAC). Στο μοντέλο αυτό, οι άδειες (permissions) δεν αποδίδονται σε μεμονωμένα άτομα αλλά σε συγκεκριμένους ρόλους (π.χ. Πελάτης, Υπάλληλος, Διαχειριστής), διασφαλίζοντας ότι κάθε χρήστης εκτελεί μόνο τις ενέργειες που επιτρέπονται από τη θέση του. Για παράδειγμα, στο προς σχεδίαση σύστημα ένας υπάλληλος μπορεί να διαβάζει τα στοιχεία ενός συμβολαίου για να το αξιολογήσει, αλλά μόνο ο πελάτης έχει το δικαίωμα να «υπογράψει ψηφιακά» την τελική αλλαγή των προσωπικών του δεδομένων.

3. Εφαρμογή της Μεθοδολογίας Μαλακών Συστημάτων (ΜΜΣ)

3.1. Περιγραφή της Προβληματικής Κατάστασης (Unstructured Problem Situation)

Η υφιστάμενη κατάσταση στην ασφαλιστική εταιρεία χαρακτηρίζεται από έλλειψη ψηφιακής αυτονομίας για τους ασφαλισμένους. Παρόλο που η εταιρεία διαθέτει εσωτερικά μηχανογραφικά συστήματα (Back-office), δεν υπάρχει διασύνδεση με τους τελικούς πελάτες (Front-end).

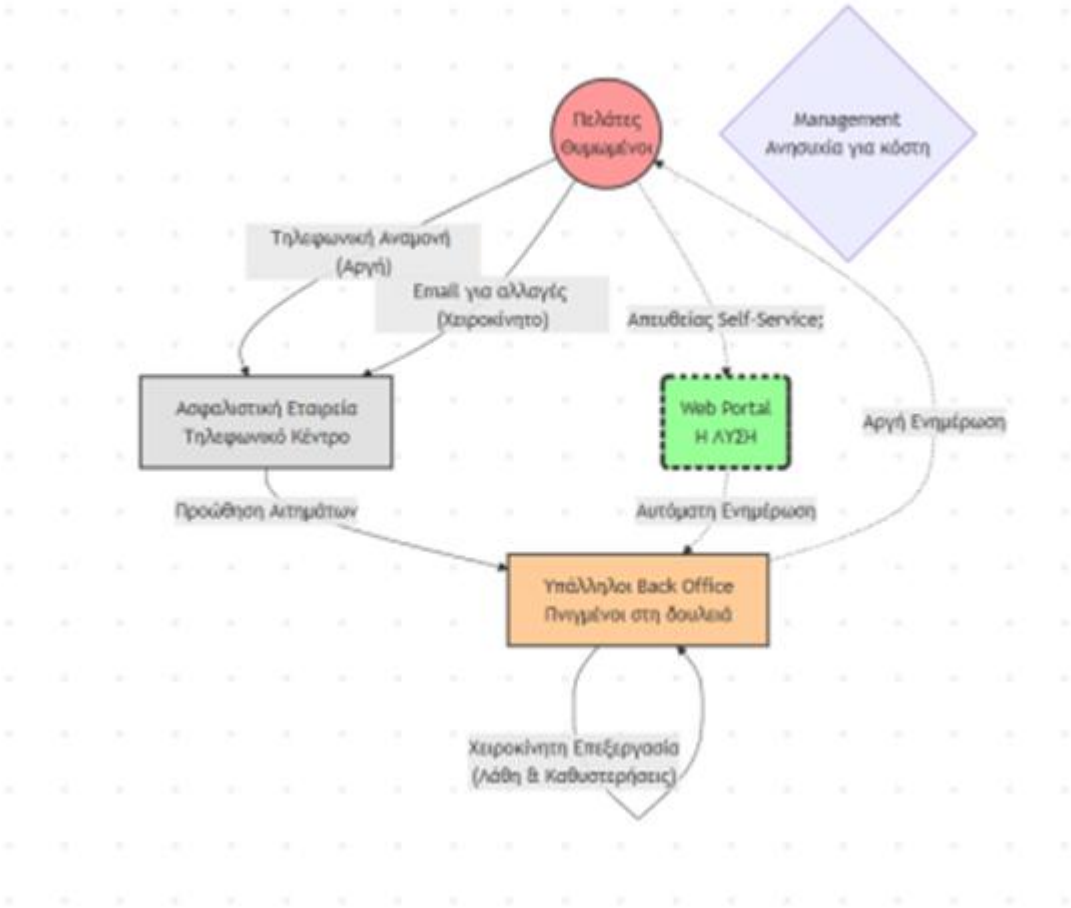
Συγκεκριμένα, παρατηρούνται τα εξής φαινόμενα που συνθέτουν την προβληματική κατάσταση:

- Συμφόρηση Επικοινωνίας: Οι τηλεφωνικές γραμμές εξυπηρέτησης υπερφορτώνονται από απλά αιτήματα (π.χ. «πότε λήγει το συμβόλαιό μου;» ή «θέλω να αλλάξω διεύθυνση»), με αποτέλεσμα μεγάλο χρόνο αναμονής και δυσαρέσκεια των πελατών.
- Χειροκίνητες Διαδικασίες: Οι τροποποιήσεις συμβολαίων (π.χ. προσθήκη οδικής βοήθειας) απαιτούν ανταλλαγή email, εκτύπωση φορμών, υπογραφή και σάρωση, διαδικασία που μπορεί να διαρκέσει ημέρες, αυξάνοντας το λειτουργικό κόστος και την πιθανότητα σφαλμάτων.
- Έλλειψη Οικονομικής Ευελιξίας: Οι πληρωμές γίνονται συχνά μέσω τραπέζης με κωδικό πληρωμής, χωρίς δυνατότητα άμεσης ενεργοποίησης/απενεργοποίησης πάγιας εντολής από τον πελάτη, οδηγώντας συχνά σε ακούσιες ακυρώσεις συμβολαίων λόγω καθυστέρησης πληρωμής, γεγονός που πλήττει τα έσοδα της εταιρείας και την αξιοπιστία της.

3.2. Η Πλούσια Εικόνα (Rich Picture)

Η «Πλούσια Εικόνα» αποτελεί την οπτική απεικόνιση της προβληματικής κατάστασης, αποτυπώνοντας τους εμπλεκόμενους, τις διεργασίες και τις συγκρούσεις. Διακρίνονται οι βασικοί δρώντες (Πελάτες, Υπάλληλοι, Διοίκηση) και οι μεταξύ τους αλληλεπιδράσεις που οδηγούν σε λειτουργική συμφόρηση. Το προτεινόμενο Web Portal εισάγεται ως το μέσο ψηφιακού μετασχηματισμού που

στοχεύει στην αποσυμφόρηση του τηλεφωνικού κέντρου και την άμεση εξυπηρέτηση του ασφαλισμένου.



Εικόνα 1: Πλούσια Εικόνα της υφιστάμενης προβληματικής κατάστασης στην Ασφαλιστική Εταιρεία.

3.3. Ανάλυση CATWOE

Για τον προσδιορισμό του συστήματος που θα επιλύσει το πρόβλημα, εφαρμόζεται η ανάλυση CATWOE, η οποία ορίζει τα βασικά στοιχεία της προτεινόμενης αλλαγής.

- C - Customers (Πελάτες): Οι άμεσα ωφελούμενοι είναι οι ασφαλισμένοι της εταιρείας (κάτοχοι συμβολαίων Αυτοκινήτου, Υγείας, Ζωής, Κατοικίας), οι οποίοι θα αποκτήσουν έλεγχο στα συμβόλαιά τους.
- A - Actors (Δράστες):
  - Οι ίδιοι οι πελάτες (που θα εκτελούν τις αλλαγές και τις πληρωμές).
  - Οι διαχειριστές του συστήματος (π.χ. IT τμήμα) που θα συντηρούν την πλατφόρμα.
- T - Transformation Process (Διεργασία Μετασχηματισμού):
  - Είσοδος: Ανάγκη πελάτη για αλλαγή στοιχείων/κάλυψης ή πληρωμή -> Χειροκίνητη/Τηλεφωνική εξυπηρέτηση.
  - Μετασχηματισμός: Αυτοματοποίηση μέσω διαδικτυακής πλατφόρμας.
  - Έξοδος: Ολοκληρωμένη συναλλαγή/τροποποίηση σε πραγματικό χρόνο χωρίς ανθρώπινη παρέμβαση (Self-Service).

- W - Worldview (Κοσμοθεωρία): Η πεποίθηση ότι η παροχή ψηφιακών εργαλείων αυτοεξυπηρέτησης αυξάνει την ικανοποίηση του πελάτη, μειώνει τα λειτουργικά έξοδα και εκσυγχρονίζει την εικόνα της εταιρείας στην αγορά.
- O - Owner (Ιδιοκτήτης): Η Διοίκηση της Ασφαλιστικής Εταιρείας, η οποία έχει την εξουσία να εγκρίνει, να χρηματοδοτήσει ή να διακόψει τη λειτουργία του συστήματος.
- E - Environmental Constraints (Περιβαλλοντικοί Περιορισμοί):
  - Νομικό πλαίσιο προστασίας προσωπικών δεδομένων (GDPR).
  - Κανονισμοί Τράπεζας της Ελλάδος (για τις ασφαλιστικές εργασίες).
  - Απαιτήσεις ασφάλειας ηλεκτρονικών πληρωμών (PSD2).
  - Ανάγκη διασύνδεσης (integration) με τα υφιστάμενα κεντρικά συστήματα (Legacy systems) της εταιρείας μέσω REST APIs.

3.4. Ορισμός Ρίζας (Root Definition)

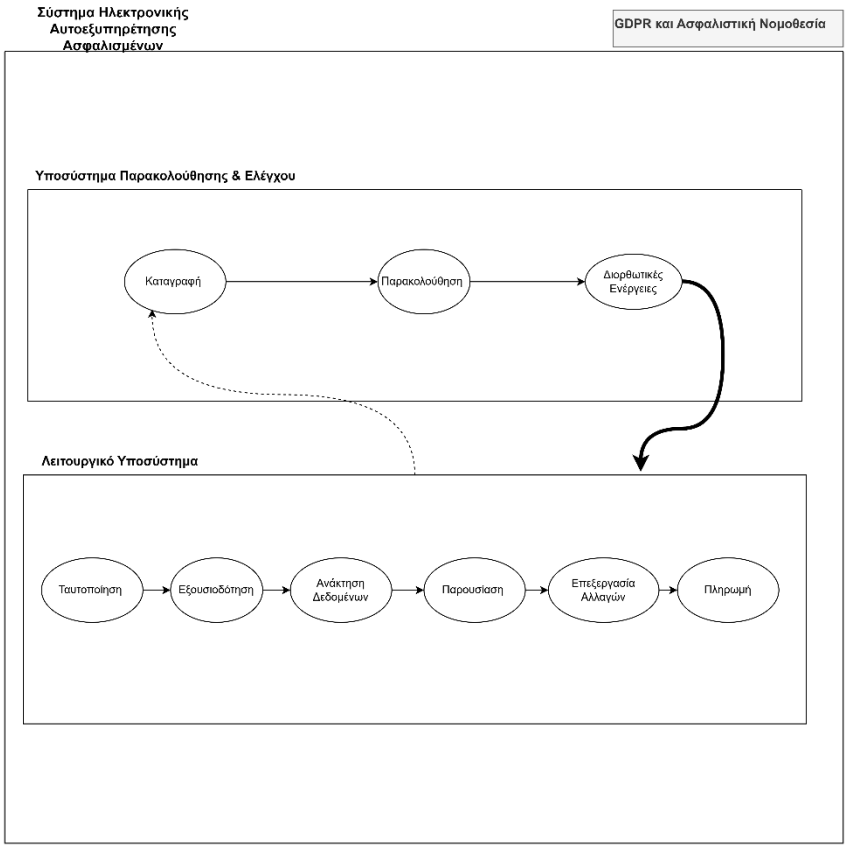
Βασιζόμενοι στην ανάλυση CATWOE, διατυπώνουμε τον Ορισμό Ρίζας του συστήματος:

«Ένα σύστημα που ανήκει στη Διοίκηση της Ασφαλιστικής Εταιρείας και λειτουργεί υπό τους περιορισμούς του GDPR και της ασφαλιστικής νομοθεσίας. Το σύστημα διαχειρίζονται οι πελάτες και το τμήμα πληροφορικής, με σκοπό να μετασχηματίσει τη διαδικασία διαχείρισης ασφαλιστηρίων από μια χρονοβόρα, χειροκίνητη εργασία σε μια άμεση, ηλεκτρονική υπηρεσία αυτοεξυπηρέτησης. Αυτό επιτυγχάνεται προκειμένου να βελτιωθεί η εμπειρία του πελάτη και η λειτουργική αποδοτικότητα της εταιρείας».

3.5. Εννοιολογικό Μοντέλο (Conceptual Model)

Με βάση τον Ορισμό Ρίζας, το σύστημα πρέπει να είναι ικανό να εκτελεί τις ακόλουθες ελάχιστες δραστηριότητες (Minimum Necessary Activities):

1. Ταυτοποίηση του χρήστη με ασφαλή τρόπο.
2. Επαλήθευση δικαιωμάτων πρόσβασης στα δεδομένα (Role-Based Access Control).
3. Ανάκτηση δεδομένων συμβολαίων από την κεντρική βάση της εταιρείας.
4. Παρουσίαση πληροφοριών ανά κλάδο (Υγεία, Αυτοκίνητο, κ.λπ.).
5. Επεξεργασία αιτημάτων αλλαγής στοιχείων και τροποποίησης καλύψεων.
6. Εκτέλεση και επιβεβαίωση ηλεκτρονικών πληρωμών.
7. Καταγραφή των ενεργειών για λόγους ελέγχου και ασφάλειας.
8. Παρακολούθηση του συστήματος, μέτρηση της απόδοσής του και της ικανοποίησης των χρηστών.
9. Διορθωτικές ενέργειες, προσαρμογή των λειτουργιών βάσει των αποτελεσμάτων της παρακολούθησης.



Εικόνα 2: Εννοιολογικό μοντέλο

Για να διασφαλιστεί ότι ο μετασχηματισμός επιτυγχάνεται, το υποσύστημα ελέγχου αξιολογεί τη λειτουργία με βάση τα εξής τρία κριτήρια:

- Αποτελεσματικότητα (Efficacy) δηλαδή κατά πόσο το σύστημα επιτρέπει στον πελάτη να ολοκληρώσει τη διαχείριση του συμβολαίου του online χωρίς ανάγκη ανθρώπινης παρέμβασης.
- Αποδοτικότητα (Efficiency) δηλαδή ποια είναι η μείωση του χρόνου και των πόρων που απαιτούνται για την επεξεργασία ενός αιτήματος σε σχέση με την προηγούμενη χειροκίνητη διαδικασία.
- Σκοπιμότητα (Effectiveness) δηλαδή ο βαθμός στον οποίο το σύστημα βελτιώνει τη συνολική εμπειρία του πελάτη και συμβάλλει στη στρατηγική ψηφιοποίησης της εταιρείας.

3.6. Σύγκριση Εννοιολογικού Μοντέλου με την Υφιστάμενη Κατάσταση

Σε αυτό το στάδιο της ΜΜΣ, συγκρίνουμε τις δραστηριότητες του ιδανικού μοντέλου (3.5) με την τρέχουσα πραγματικότητα (3.1) για να εντοπίσουμε τις αναγκαίες αλλαγές:

Δραστηριότητα Μοντέλου	**Υφιστάμενη Κατάσταση**	**Προτεινόμενη Αλλαγή (Μέσω του ΠΣ)**
Ταυτοποίηση & Εξουσιοδότηση	Χειροκίνητη επαλήθευση μέσω τηλεφώνου/email.	Αυτοματοποιημένο Login και RBAC έλεγχος.
Παρουσίαση Πληροφοριών	Ο πελάτης δεν έχει άμεση πρόσβαση στα δεδομένα του.	Real-time Dashboard με ανάλυση ανά κλάδο.

Επεξεργασία Αλλαγών	Χρονοβόρα ανταλλαγή εγγράφων και χειροκίνητη καταχώρηση.	Ψηφιακή φόρμα με άμεσο υπολογισμό ασφαλίστρου.
Εκτέλεση Πληρωμών	Μόνο μέσω τραπέζης (offline) ή τηλεφώνου.	Ενσωματωμένη πύλη πληρωμών (Payment Gateway).

Πίνακας 1: Σύγκριση Εννοιολογικού Μοντέλου με την Υφιστάμενη Κατάσταση

4. Διαδικασία Ανάπτυξης Συστήματος και Ανάλυση

4.1. Επιλογή και Αιτιολόγηση Μεθοδολογίας Agile (SCRUM)

Για την ανάπτυξη του Πληροφοριακού Συστήματος της Ασφαλιστικής Εταιρείας επιλέχθηκε το πλαίσιο εργασίας SCRUM, το οποίο εντάσσεται στην οικογένεια των Agile (Ευέλικτων) μεθοδολογιών. Η επιλογή αυτή δεν είναι τυχαία, αλλά προκύπτει από τη φύση της προβληματικής κατάστασης που αναλύθηκε στο Κεφάλαιο 3.
Συγκεκριμένα, το έργο χαρακτηρίζεται από:

1. Αβεβαιότητα Απαιτήσεων: Οι ανάγκες των πελατών για το πώς θέλουν να βλέπουν τα συμβόλαιά τους μπορεί να αλλάξουν κατά τη διάρκεια της ανάπτυξης.
2. Πολυπλοκότητα: Η επιχειρησιακή λογική (π.χ. υπολογισμός ασφαλίστρων - underwriting) είναι σύνθετη και απαιτεί δοκιμές.
3. Ανάγκη για Γρήγορα Αποτελέσματα: Η εταιρεία χρειάζεται άμεσα μια λειτουργική έκδοση (MVP) για να αποσυμφορήσει το τηλεφωνικό κέντρο.

Το SCRUM επιτρέπει την τμηματική και επαναληπτική (iterative and incremental) ανάπτυξη, προσφέροντας παραδοτέα σε μικρούς κύκλους (Sprints) των 2-4 εβδομάδων.

4.1.1. Συγκριτική Ανάλυση: Waterfall vs Agile-SCRUM

Για την περαιτέρω τεκμηρίωση της επιλογής, παρατίθεται σύγκριση μεταξύ του παραδοσιακού μοντέλου Καταρράκτη (Waterfall) και του SCRUM.

Χαρακτηριστικό	**Παραδοσιακό Μοντέλο (Waterfall)**	**Agile Μοντέλο (SCRUM)**	**Γιατί επιλέχθηκε το SCRUM;**
Διαχείριση Αλλαγών	Δύσκαμπτη. Οι απαιτήσεις «κλειδώνουν» στην αρχή. Αλλαγές κοστίζουν ακριβά.	Ευέλικτη. Οι αλλαγές είναι ευπρόσδεκτες σε κάθε νέο Sprint.	Στο Web Portal, οι χρήστες μπορεί να ζητήσουν νέες λειτουργίες (π.χ. πληρωμή με PayPal) που δεν είχαν προβλεφθεί αρχικά.
Παράδοση Αξίας	Το λογισμικό παραδίδεται ολόκληρο στο τέλος του έργου (Big Bang).	Συνεχής παράδοση λειτουργικών κομματιών (π.χ. πρώτα το Προφίλ, μετά οι Πληρωμές).	Η εταιρεία μπορεί να ξεκινήσει να χρησιμοποιεί το σύστημα νωρίτερα, μειώνοντας άμεσα τον φόρτο εργασίας.

Εμπλοκή Πελάτη	Ελάχιστη κατά την υλοποίηση. Ο πελάτης βλέπει το προϊόν στο τέλος.	Υψηλή. Ο Product Owner (εκπρόσωπος πελάτη) συμμετέχει συνεχώς και δίνει feedback.	Εξασφαλίζεται ότι το τελικό προϊόν καλύπτει τις πραγματικές ανάγκες των ασφαλισμένων (User Centric).
Διαχείριση Ρίσκου	Υψηλό ρίσκο αποτυχίας στο τέλος αν κάτι σχεδιάστηκε λάθος.	Το ρίσκο επιμερίζεται. Τα λάθη εντοπίζονται και διορθώνονται νωρίς (Fail Fast).	Αποφυγή σφαλμάτων σε κρίσιμες λειτουργίες όπως οι οικονομικές συναλλαγές.

Πίνακας 2: Σύγκριση Μεθοδολογιών Agile και Waterfall

4.1.2. Σύγκριση με άλλες Agile Μεθοδολογίες (Kanban)

Σε αντίθεση με το Kanban, το οποίο εστιάζει στη συνεχή ροή εργασιών χωρίς χρονικά οριοθετημένους κύκλους, το SCRUM επιλέχθηκε διότι παρέχει αυστηρό χρονοδιάγραμμα (Time-boxing) μέσω των Sprints. Αυτό είναι κρίσιμο για την τήρηση των προθεσμιών της εργασίας και την οργάνωση των παραδοτέων σε σαφείς φάσεις (Σύλληψη, Ορισμός, Εκτέλεση), όπως απαιτείται από τις προδιαγραφές του έργου.

4.1.3. Ρόλοι Scrum

- Product Owner: Εκπρόσωπος της Ασφαλιστικής Εταιρείας (καθορίζει τις προτεραιότητες).
- Scrum Master: Ο υπεύθυνος για την τήρηση της διαδικασίας (ερευνητής/φοιτητής).
- Development Team: Η ομάδα ανάπτυξης λογισμικού (ερευνητής/φοιτητής).

4.2. Φάσεις Διαδικασίας Ανάπτυξης

Η διαδικασία ανάπτυξης χαρτογραφείται στις τρεις θεμελιώδεις φάσεις ενός έργου ΠΣ:

1. Φάση Σύλληψης (Conception Phase):
   - Περιλαμβάνει τον ορισμό του οράματος (Vision) και την αρχική ανάλυση απαιτήσεων μέσω της μεθοδολογίας SSM (όπως αναλύθηκε στο Κεφάλαιο 3).
   - Παραδοτέο: Product Backlog (Λίστα Απαιτήσεων).
2. Φάση Ορισμού (Definition Phase - Sprint 0):
   - Επιλογή τεχνολογιών (Python/Java, SQL).
   - Σχεδιασμός Αρχιτεκτονικής.
   - Δημιουργία διαγραμμάτων UML.
   - Παραδοτέο: Αρχιτεκτονικό Σχέδιο και Περιβάλλον Ανάπτυξης.
3. Φάση Εκτέλεσης (Execution Phase - Sprints):
   - Υλοποίηση των λειτουργιών σε κύκλους (Sprints) διάρκειας 2 εβδομάδων.
   - Sprint 1: Βάση Δεδομένων & Authentication.
   - Sprint 2: Προβολή Χαρτοφυλακίου & Στοιχεία Πελάτη.
   - Sprint 3: Διαχείριση Πληρωμών & Τροποποιήσεις Συμβολαίων.
   - Παραδοτέο: Λειτουργικό Λογισμικό (Potentially Shippable Product).

4.3. Product Backlog (Λίστα Απαιτήσεων)

4.3.1. Προτεραιοποίηση Απαιτήσεων (MoSCoW Method)

Για την ιεράρχηση των ιστοριών χρήστη στο Product Backlog εφαρμόστηκε η μέθοδος MoSCoW (Must have, Should have, Could have, Won't have).

• Must have (Επιβεβλημένα): Η ταυτοποίηση, η προβολή συμβολαίων και η πληρωμή ορίστηκαν ως ύψιστης προτεραιότητας, καθώς χωρίς αυτά το σύστημα δεν έχει λειτουργική αξία για την αποσυμφόρηση του τηλεφωνικού κέντρου.

• Should have (Σημαντικά): Η τροποποίηση καλύψεων (Underwriting) εντάχθηκε σε δεύτερη μοίρα, ως λειτουργία προστιθέμενης αξίας.

• Could have (Επιθυμητά): Λειτουργίες όπως η «Δήλωση Ατυχήματος» ή το «Chatbot» μετατέθηκαν για μελλοντικά Sprints (βλ. Μελλοντικές Επεκτάσεις), ώστε να διασφαλιστεί η έγκαιρη παράδοση του βασικού συστήματος (MVP).

4.3.2. Καταγραφή Ιστοριών Χρήστη (User Stories).

Ακολουθεί η καταγραφή των ιστοριών χρήστη (User Stories) που συνθέτουν το Product Backlog:

ID	User Story (Ως πελάτης θέλω να...)	Κριτήρια Αποδοχής
US01	...βλέπω τα συμβόλαιά μου ανά κλάδο.	Εμφάνιση καρτελών: Αυτοκίνητο, Υγεία, Ζωή, Κατοικία.
US02	...τροποποιώ τα προσωπικά μου στοιχεία.	Φόρμα αλλαγής Διεύθυνσης, Επαγγέλματος, ΑΔΤ.
US03	...βλέπω τις ημερομηνίες ανανέωσης.	Εμφάνιση ημερομηνίας λήξης και ποσού επόμενης δόσης.
US04	...διαχειρίζομαι τις πληρωμές μου.	Επιλογή: Πάγια Εντολή (On/Off), Συχνότητα (Μήνα/3μηνο/6μηνο/Έτος).
US05	...πληρώνω online.	Διασύνδεση με φόρμα πληρωμής κάρτας.

US06	...αλλάζω τις παροχές του συμβολαίου μου.	Προσθήκη/Αφαίρεση καλύψεων (π.χ. Οδική Βοήθεια) με live επανυπολογισμό ασφαλίστρου.

Πίνακας 3: Καταγραφή User Stories

4.4. Προγραμματισμός Sprints και Story Points

Η υλοποίηση του συστήματος οργανώθηκε σε τρεις επαναληπτικούς κύκλους (Sprints). Για την εκτίμηση της προσπάθειας χρησιμοποιήθηκε η μονάδα Story Points (βάσει της κλίμακας Fibonacci), η οποία εκφράζει την πολυπλοκότητα κάθε εργασίας.

Sprint	Στόχος & Παραδοτέα	Story Points (Πολυπλοκότητα)
Sprint 1	Ανάπτυξη Backend (Java Entities & Controllers) και Βάσης Δεδομένων.	**13**
Sprint 2	Ενορχήστρωση Workflow στο Bonita Studio και σύνδεση μέσω REST Connectors.	**21**
Sprint 3	Σχεδιασμός UI φορμών και Containerization με Docker.	**8**

Πίνακας 4: Παρουσίαση Επαναληπτικών Κύκλων Ανάπτυξης

- Sprint 1 (13 points): Μέση δυσκολία, καθώς αφορούσε την κατασκευή των θεμελιωδών δομών της Java.
- Sprint 2 (21 points): Αποτελεί το πιο σύνθετο κομμάτι, καθώς η διασύνδεση (integration) της μηχανής Bonita με το Spring Boot API απαιτεί ακρίβεια στη ρύθμιση των REST Connectors.
- Sprint 3 (8 points): Εστιάζει στην τελική οπτικοποίηση και τη διάθεση (Deployment) μέσω Docker, έχοντας μικρότερο βαθμό αβεβαιότητας.

Η τήρηση των τελετουργικών (Scrum Events) είναι καθοριστική για τη διασφάλιση της ποιότητας των παραδοτέων σχεδιασμού:
Daily Stand-up: Σε καθημερινή βάση, η συνάντηση αυτή χρησιμοποιείται για τον συντονισμό της μοντελοποίησης, επιλύοντας άμεσα ασάφειες που προκύπτουν κατά τη χαρτογράφηση των δεδομένων και των διεργασιών.

Sprint Review: Με την ολοκλήρωση κάθε σχεδιαστικού κύκλου, το Review δίνει τη δυνατότητα παρουσίασης των παραδοτέων (όπως τα διαγράμματα UML ή ο σχεδιασμός της βάσης δεδομένων) στον Product Owner για έγκριση. Για παράδειγμα, στο τέλος του Sprint 2, η παρουσίαση του σχεδιασμού του workflow στο Bonita Studio επιτρέπει την άμεση ανατροφοδότηση και τη διόρθωση της επιχειρησιακής λογικής πριν προχωρήσει ο σχεδιασμός των φορμών.

Sprint Retrospective: Τέλος, η διαδικασία αυτή εστιάζει στην αξιολόγηση των εργαλείων μοντελοποίησης που χρησιμοποιούνται, διασφαλίζοντας ότι η μετάβαση

από το Εννοιολογικό Μοντέλο της ΜΜΣ στον τεχνικό σχεδιασμό του Πληροφοριακού Συστήματος πραγματοποιείται με τη μέγιστη δυνατή ακρίβεια.

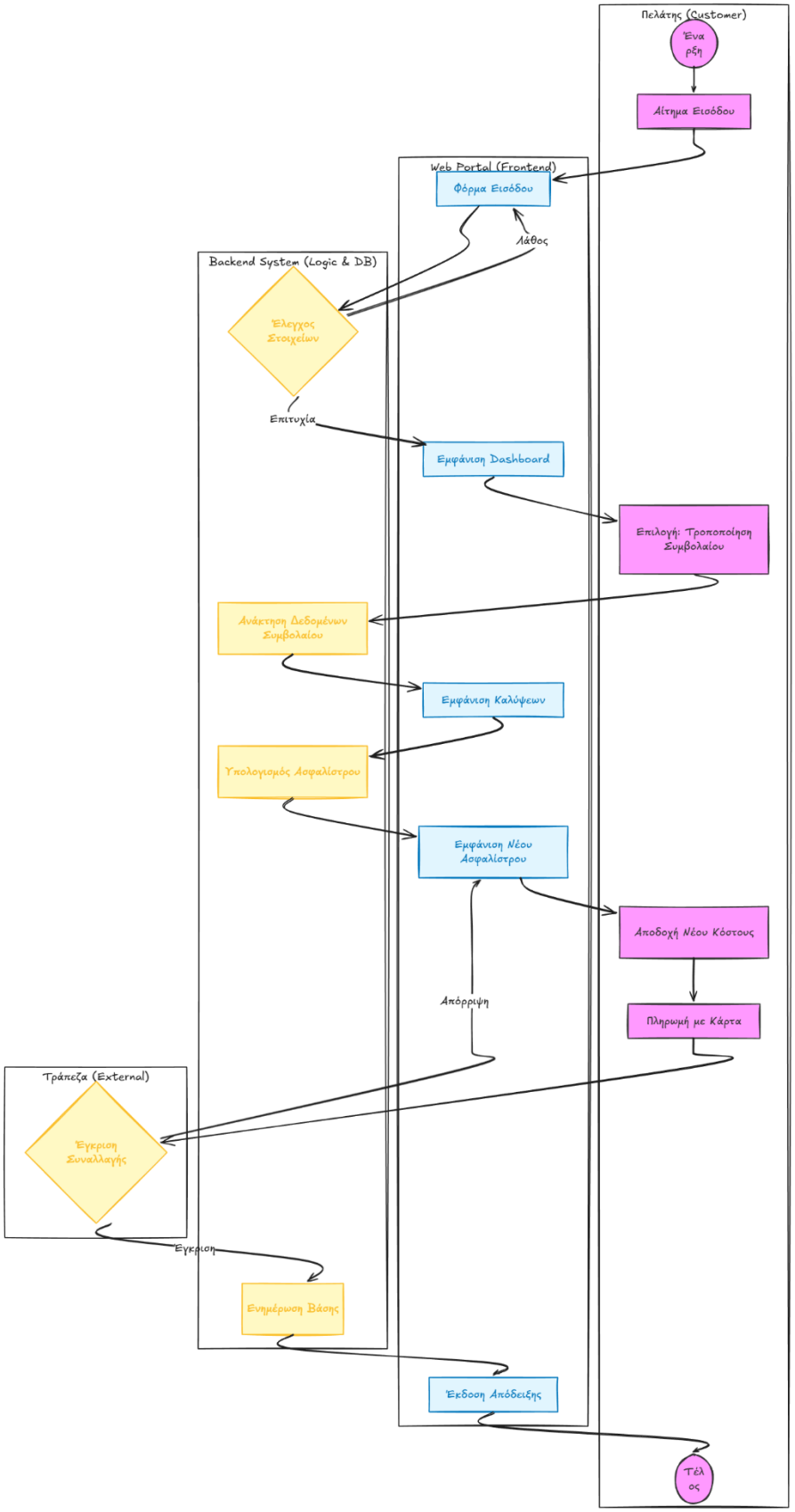
Για την περιγραφή των στατικών και δυναμικών χαρακτηριστικών του συστήματος χρησιμοποιείται η γλώσσα UML.

4.5. Μοντελοποίηση Συστήματος

4.5.1. Μοντελοποίηση Επιχειρησιακών Διεργασιών (BPMN)

Πριν την ανάλυση των τεχνικών προδιαγραφών, κρίνεται απαραίτητη η χαρτογράφηση της συνολικής επιχειρησιακής ροής (Business Process) για τη βασική λειτουργία του συστήματος. Το Διάγραμμα Ροής αποτυπώνει τη διαστρωμάτωση των εργασιών σε διακριτές ζώνες ευθύνης (Swimlanes): τον Πελάτη, το Περιβάλλον Χρήστη (Frontend), το Κεντρικό Σύστημα (Backend/Database) και τα Εξωτερικά Συστήματα (Τράπεζα).

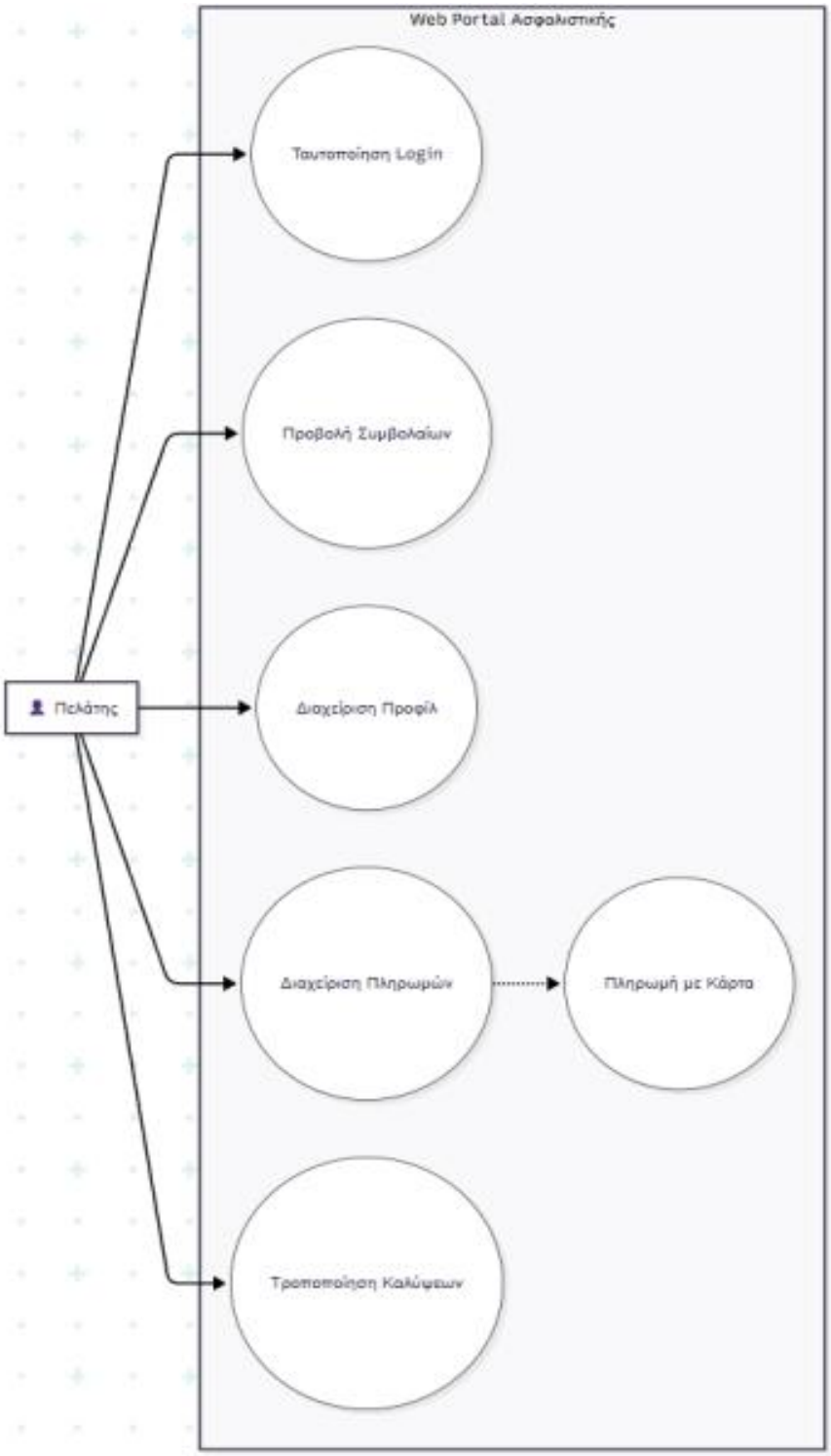
Όπως φαίνεται στο ακόλουθο διάγραμμα, η διαδικασία ξεκινά με την ταυτοποίηση του πελάτη. Κάθε ενέργεια του χρήστη (π.χ. επιλογή κάλυψης) πυροδοτεί μια διεργασία στο Backend (υπολογισμός ασφαλίστρου), η οποία επιστρέφει αποτελέσματα στο Frontend. Η διαδικασία ολοκληρώνεται μόνο εφόσον ληφθεί έγκριση από το εξωτερικό τραπεζικό σύστημα, διασφαλίζοντας την οικονομική ακεραιότητα της συναλλαγής πριν την οριστική ενημέρωση της βάσης δεδομένων.



Εικόνα 3: Μοντελοποίηση της διεργασίας αυτοεξυπηρέτησης σε περιβάλλον SOA με χρήση διακριτών ζωνών ευθύνης (Swimlanes)

4.5.2. Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram)

Το διάγραμμα αυτό περιγράφει τις αλληλεπιδράσεις του χρήστη με το σύστημα.



Εικόνα 4: Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram) του Web Portal Ασφαλιστικής

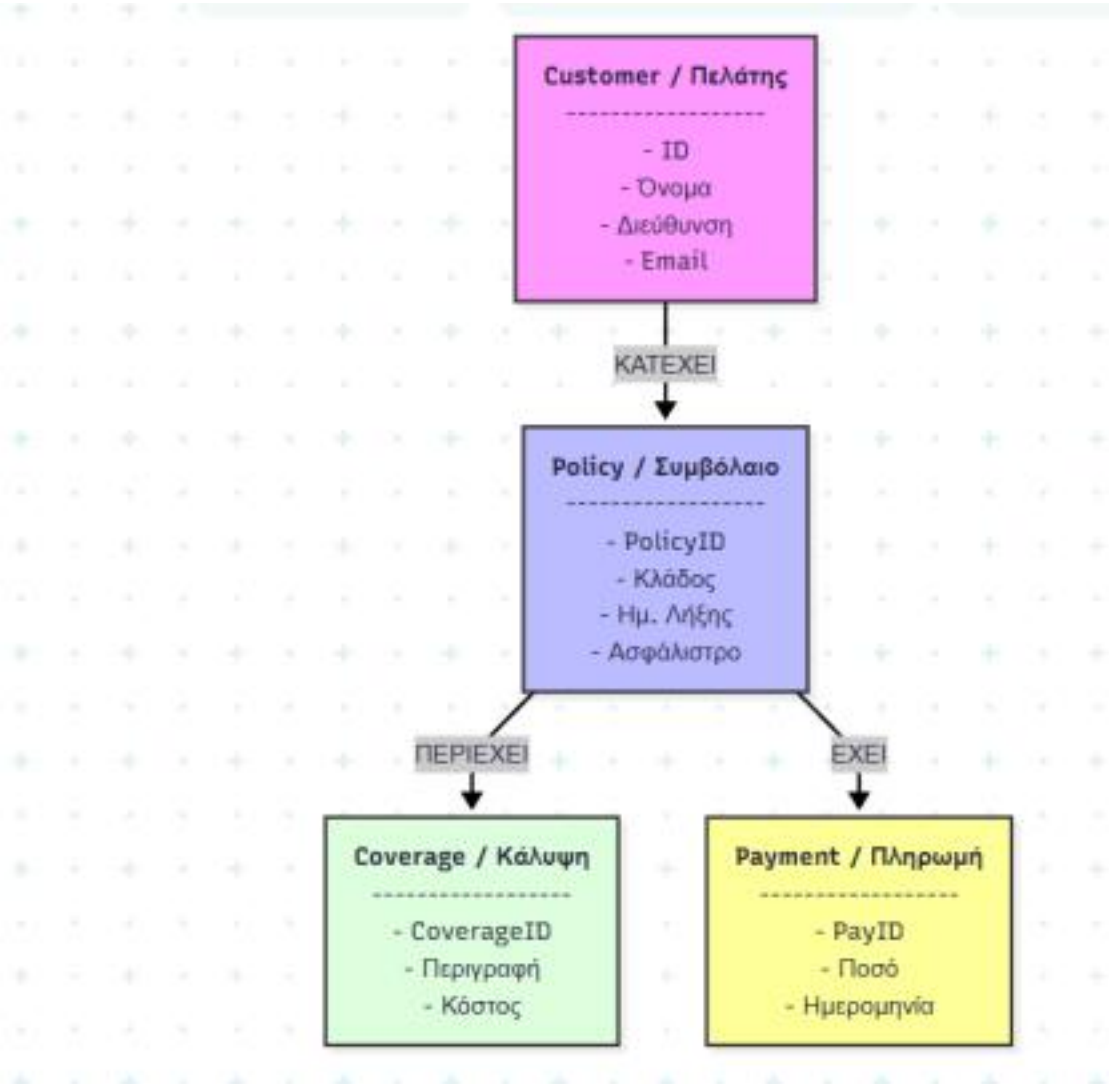
Σενάριο Χρήσης «Ηλεκτρονική Πληρωμή»

Πεδίο	Περιγραφή
Περίπτωση Χρήσης	UC-05: Ηλεκτρονική Πληρωμή
Πρωτεύων Χρήστης	Πελάτης (Ασφαλισμένος)
Προϋποθέσεις	Ο χρήστης έχει συνδεθεί και έχει επιλέξει συμβόλαιο με οφειλή.
Βασική Ροή (Happy Path)	1. Ο Χρήστης επιλέγει «Πληρωμή».
	2. Το Σύστημα ζητά στοιχεία κάρτας.
	3. Ο Χρήστης εισάγει τα στοιχεία και πατάει «Επιβεβαίωση».
	4. Το Σύστημα επικοινωνεί με την Τράπεζα.
	5. Η Τράπεζα εγκρίνει τη συναλλαγή.
	6. Το Σύστημα εμφανίζει μήνυμα επιτυχίας και εκδίδει απόδειξη.
Εναλλακτικές Ροές (Exceptions)	**3α. Ακύρωση από Χρήστη:** Ο χρήστης πατάει «Άκυρο». Το σύστημα επιστρέφει στο Dashboard.
	5α. Απόρριψη Κάρτας: Η Τράπεζα απορρίπτει τη συναλλαγή (π.χ. ανεπαρκές υπόλοιπο). Το σύστημα εμφανίζει μήνυμα «Η συναλλαγή απορρίφθηκε» και ζητά νέα κάρτα.
	5β. Τεχνικό Σφάλμα (Timeout): Το σύστημα της Τράπεζας δεν απαντά. Το σύστημα εμφανίζει «Προσωρινό πρόβλημα, δοκιμάστε αργότερα».

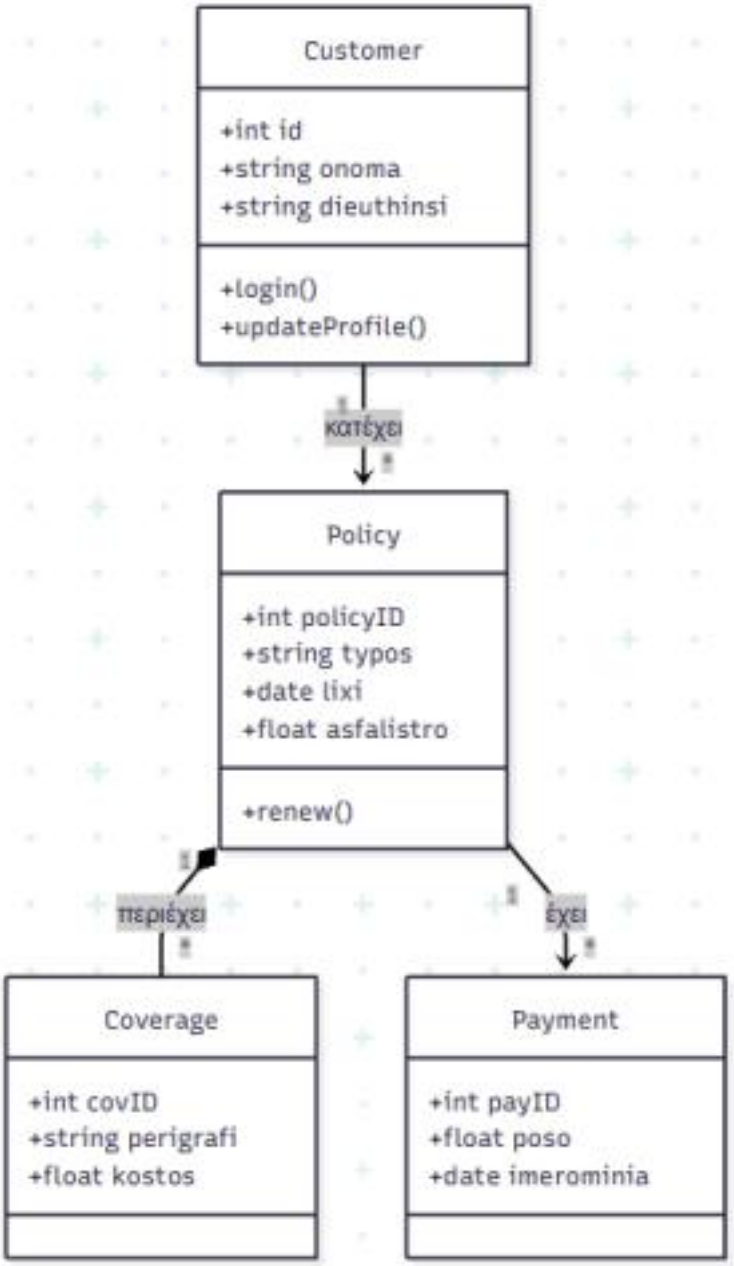
Πίνακας 5: Σενάριο Χρήσης- Ηλεκτρονική Πληρωμή

4.5.3. Διάγραμμα Κλάσεων (Class Diagram)

Το Διάγραμμα Κλάσεων αποτυπώνει τη στατική δομή του συστήματος και τη σχέση μεταξύ των οντοτήτων της βάσης δεδομένων. Η κεντρική κλάση Customer (Πελάτης) συνδέεται με πολλαπλά αντικείμενα τύπου Policy (Συμβόλαιο), καθώς ένας πελάτης μπορεί να διαθέτει ασφάλειες σε διαφορετικούς κλάδους (Υγείας, Αυτοκινήτου κ.ά.). Κάθε Συμβόλαιο αποτελείται από (σχέση Composition) μία ή περισσότερες Coverages (Καλύψεις) και συνδέεται με το ιστορικό Payments (Πληρωμών) του.



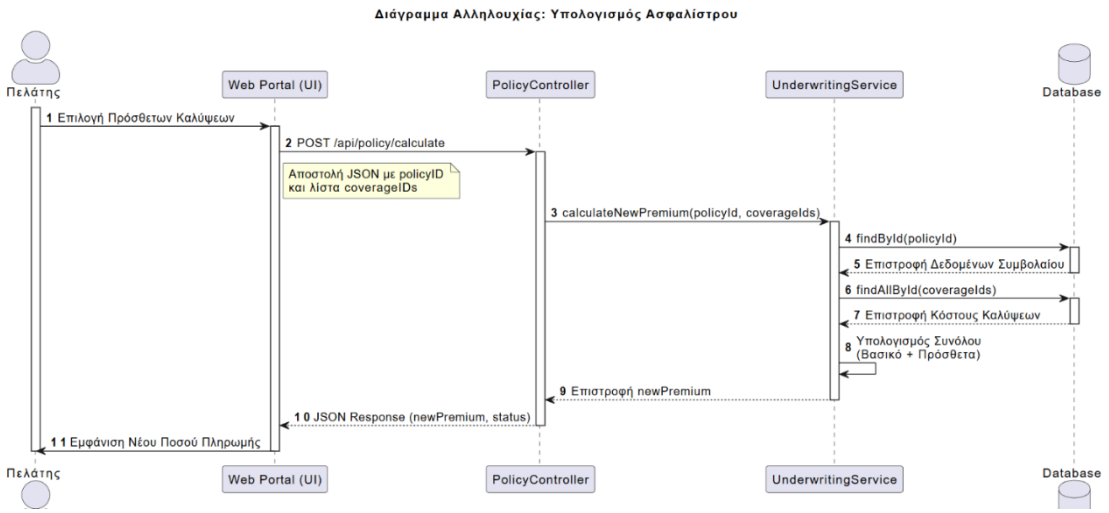
Εικόνα 5: Εννοιολογικό μοντέλο (Conceptual Model) των βασικών οντοτήτων του συστήματος.



Εικόνα 6: Αναλυτικό Διάγραμμα Κλάσεων (UML Class Diagram) με τύπους δεδομένων και λειτουργικές μεθόδους.

4.5.4. Διάγραμμα Αλληλουχίας (Sequence Diagram)

Το παρακάτω διάγραμμα αλληλουχίας αποτυπώνει τη δυναμική συμπεριφορά του συστήματος κατά τη διαδικασία υπολογισμού του ασφαλίστρου (Underwriting). Συγκεκριμένα, παρουσιάζει την αλληλεπίδραση και τη ροή μηνυμάτων μεταξύ του χρήστη, της διεπαφής (Web Portal), του Ελεγκτή (PolicyController), της Υπηρεσίας Επιχειρησιακής Λογικής (UnderwritingService) και της Βάσης Δεδομένων.



Εικόνα 7: Διάγραμμα Αλληλουχίας (Sequence Diagram) για τη δυναμική διαδικασία υπολογισμού νέου ασφαλίστρου (Underwriting)

Όπως φαίνεται, η ροή ξεκινά με το αίτημα του πελάτη για προσθήκη καλύψεων. Το αίτημα μεταφέρεται μέσω REST API κλήσης (POST /calculate) στο Backend. Εκεί, η υπηρεσία ανακτά τα δεδομένα του συμβολαίου και το κόστος των καλύψεων από τη βάση δεδομένων, εκτελεί τον υπολογισμό και επιστρέφει το τελικό ποσό στον χρήστη σε μορφή JSON.

Επεξήγηση διαγράμματος αλληλουχίας υπολογισμού ασφαλίστρου (Underwriting)

Οντότητες:

Actor: Πελάτης (User)
Object 1: Web Portal (UI)
Object 2: PolicyController (Backend API)
Object 3: UnderwritingService (Logic)
Object 4: Database (Repository)

Ροή (Βέλη):

1. Ο Πελάτης επιλέγει καλύψεις -> στέλνει αίτημα στο UI.
2. Το UI στέλνει POST /calculate στο PolicyController.
3. Ο PolicyController καλεί τη μέθοδο calculateNewPremium στο UnderwritingService.
4. Το UnderwritingService κάνει findById στη Database για να βρει το συμβόλαιο.
5. Η Database επιστρέφει τα δεδομένα συμβολαίου.
6. Το UnderwritingService προσθέτει το κόστος των νέων καλύψεων και επιστρέφει το ποσό.
7. Ο PolicyController επιστρέφει JSON απάντηση στο UI.
8. Το UI εμφανίζει τη νέα τιμή στον Πελάτη.

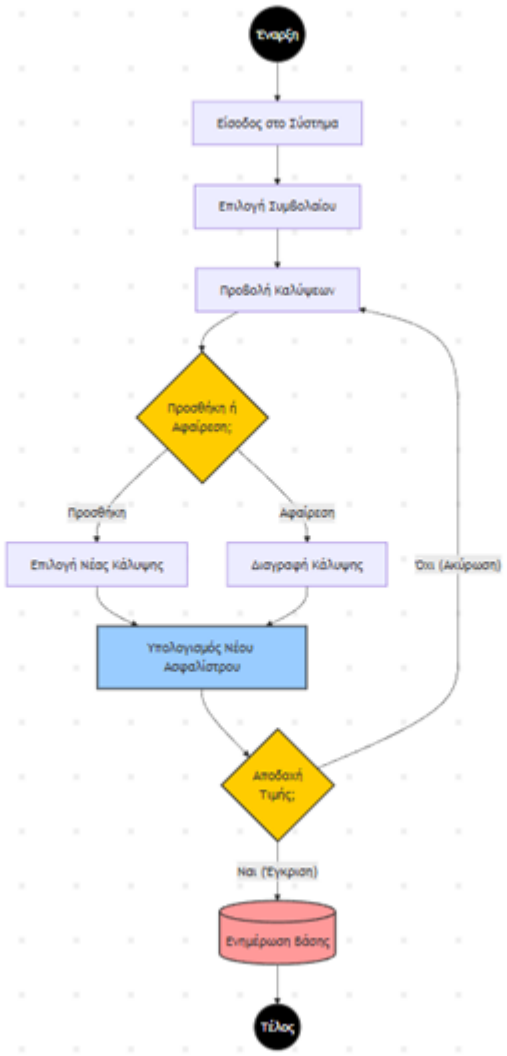
4.5.5. Διάγραμμα Δραστηριοτήτων

Τα διαγράμματα δραστηριοτήτων οπτικοποιούν τη λογική ροή των εργασιών (workflow) και τις αποφάσεις που λαμβάνονται κατά την εκτέλεση κρίσιμων διεργασιών.

4.5.5.1. Τροποποίηση Συμβολαίου και Καλύψεων

Το ακόλουθο διάγραμμα περιγράφει τη ροή εργασιών για τη διαδικασία μεταβολής των όρων ενός ενεργού ασφαλιστηρίου:

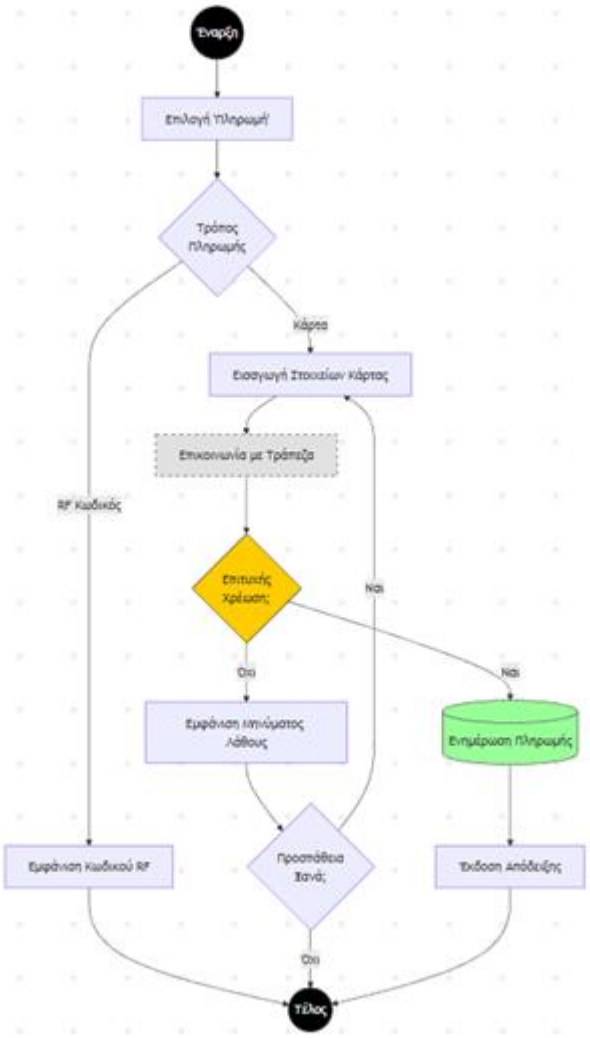
Ο χρήστης επιλέγει το επιθυμητό συμβόλαιο και μεταβαίνει στην οθόνη διαχείρισης καλύψεων. Το σύστημα επιτρέπει την προσθήκη ή αφαίρεση προαιρετικών παροχών, ενεργοποιώντας αυτόματα τον αλγόριθμο υπολογισμού ασφαλίστρου (Underwriting engine) σε κάθε αλλαγή. Εάν ο χρήστης αποδεχτεί το νέο κόστος, η αλλαγή καταγράφεται στη βάση δεδομένων, ενώ σε περίπτωση ακύρωσης, το σύστημα επιστρέφει στην αρχική κατάσταση χωρίς να μεταβάλει τα δεδομένα.



Εικόνα 8: Διάγραμμα Δραστηριοτήτων για τη διαδικασία τροποποίησης καλύψεων συμβολαίου.

4.5.5.2. Διαδικασία Ηλεκτρονικής Πληρωμής

Στο ακόλουθο διάγραμμα παρουσιάζεται η ροή της διαδικασίας πληρωμής. Η διαδικασία αυτή είναι κρίσιμη καθώς περιλαμβάνει αλληλεπίδραση με εξωτερικό τραπεζικό σύστημα. Εάν ο χρήστης επιλέξει πληρωμή με κάρτα, το σύστημα επικοινωνεί με τον πάροχο πληρωμών (Payment Gateway). Σε περίπτωση επιτυχούς συναλλαγής, ενημερώνεται άμεσα η βάση δεδομένων και εκδίδεται η απόδειξη. Σε περίπτωση απόρριψης της κάρτας, δίνεται η δυνατότητα επανάληψης της προσπάθειας.



Εικόνα 9: Διάγραμμα Δραστηριοτήτων για την ολοκλήρωση ηλεκτρονικής πληρωμής.

4.6. Απαιτήσεις Ασφάλειας και Ελέγχου Πρόσβασης (Security & Authorizations)

Σύμφωνα με τις προδιαγραφές ασφαλείας, το σύστημα πρέπει να διασφαλίζει ότι οι χρήστες έχουν πρόσβαση αποκλειστικά στα δεδομένα που τους αφορούν (Data Privacy). Στον ακόλουθο πίνακα καταγράφονται οι κανόνες εξουσιοδότησης για τον ρόλο του Πελάτη (Customer).

ID	Κανόνας Ασφαλείας (Policy)	Περιγραφή	Σχετικό User Story
SEC 01	**Πρόσβαση σε Δεδομένα**	Ο χρήστης μπορεί να βλέπει (read) **μόνο** τα συμβόλαια που συνδέονται με το δικό του Customer_ID. Απαγορεύεται η πρόσβαση σε δεδομένα άλλων πελατών.	US01, US03 (Προβολή Χαρτοφυλακίου & Ημερομηνιών)
SEC 02	**Τροποποίηση Προφίλ**	Ο χρήστης μπορεί να επεξεργάζεται (write/update) τα στοιχεία επικοινωνίας του (Διεύθυνση, Τηλέφωνο, Email).	US02 (Τροποποίηση Προσωπικών Στοιχείων)
SEC 03	**Ακεραιότητα Ταυτοποίησης**	Τα πεδία ταυτοποίησης (Ονοματεπώνυμο, ΑΦΜ) είναι **Κλειδωμένα (Read-only)**. Ο χρήστης δεν μπορεί να τα τροποποιήσει ηλεκτρονικά για λόγους αποφυγής απάτης (anti-fraud).	US02 (Περιορισμός κατά την αλλαγή στοιχείων)
SEC 04	**Διαχείριση Συμβολαίου**	Ο χρήστης μπορεί να προσθέτει/αφαιρεί καλύψεις (modify coverage) μόνο σε ενεργά συμβόλαια ιδιοκτησίας του. Δεν επιτρέπεται η αλλαγή του βασικού ασφαλίστρου (base premium) που ορίζεται από την εταιρεία.	US06 (Αλλαγή παροχών συμβολαίου)
SEC 05	**Οικονομικές Συναλλαγές**	Ο χρήστης μπορεί να εκτελεί πληρωμές και να διαχειρίζεται την πάγια εντολή μόνο για τις δικές του οφειλές.	US04, US05 (Διαχείριση & Εκτέλεση Πληρωμών)

Πίνακας 6: Μήτρα Εξουσιοδοτήσεων Χρηστών

4.7. Αρχιτεκτονική Ασφάλειας και Προστασίας Δεδομένων

Για την ολοκληρωμένη προστασία του Πληροφοριακού Συστήματος και των ευαίσθητων προσωπικών δεδομένων των ασφαλισμένων, σχεδιάστηκε μια πολυεπίπεδη αρχιτεκτονική ασφάλειας που βασίζεται στους εξής τέσσερις πυλώνες:

1. Διαχείριση Ταυτοτήτων (Identity Management - IdM): Το σύστημα υλοποιεί αυστηρό μηχανισμό διαχείρισης ψηφιακών ταυτοτήτων. Κάθε χρήστης (Ασφαλισμένος) διαθέτει μοναδικά διαπιστευτήρια εισόδου (Unique

Credentials), τα οποία συνδέονται με το Customer_ID στη βάση δεδομένων. Η διαδικασία πιστοποίησης (Authentication) διασφαλίζει ότι μόνο οι εγγεγραμμένοι χρήστες αποκτούν ψηφιακή ταυτότητα εντός του συστήματος, αποτρέποντας την πλαστοπροσωπία.

2. Εμπιστευτικότητα (Confidentiality): Διασφαλίζεται ότι τα δεδομένα είναι προσβάσιμα μόνο από εξουσιοδοτημένα άτομα.
   Σε επίπεδο δεδομένων (Data at Rest): Οι κωδικοί πρόσβασης αποθηκεύονται κρυπτογραφημένοι (Hashed) με χρήση του.
   αλγορίθμου BCrypt, καθιστώντας τους μη αναγνώσιμους ακόμα και σε περίπτωση διαρροής της βάσης δεδομένων.
   Σε επίπεδο οπτικής επαφής: Ευαίσθητα πεδία όπως το ΑΦΜ ή ο αριθμός πιστωτικής κάρτας εμφανίζονται με απόκρυψη χαρακτήρων (masking) στις οθόνες του χρήστη (π.χ. ****-****-****- 1234).
3. Ασφάλεια Μηνύματος (Message Security): Για την προστασία των δεδομένων κατά τη μεταφορά τους (Data in Transit) μεταξύ του Browser του πελάτη και του Server (Spring Boot), η αρχιτεκτονική επιβάλλει τη χρήση πρωτοκόλλου HTTPS (TLS/SSL). Αυτό εγγυάται ότι κανένα μήνυμα (αίτημα πληρωμής, αλλαγή στοιχείων) δεν μπορεί να υποκλαπεί ή να
   τροποποιηθεί από τρίτους (Man-in-the-Middle attacks) κατά τη διάρκεια της επικοινωνίας.
4. Κατανεμημένες Πολιτικές (Distributed Policies): Λόγω της υπηρεσιοστρεφούς αρχιτεκτονικής (Service Oriented Architecture - SOA) που απαιτείται, οι πολιτικές ασφαλείας δεν είναι «σκληρά κωδικιοποιημένες» (hardcoded) σε ένα σημείο, αλλά εφαρμόζονται δυναμικά μέσω του Spring Security Filter Chain. Το σύστημα διαχωρίζει τις πολιτικές ανάλογα με τον ρόλο (Role-Based Access Control):
   - Public Policy: Πρόσβαση σε Login/Register.
   - Customer Policy: Πρόσβαση σε συμβόλαια και πληρωμές.
   - Admin Policy: Πρόσβαση σε διαχειριστικές λειτουργίες. Αυτό επιτρέπει την εύκολη επέκταση των πολιτικών σε περίπτωση μελλοντικής προσθήκης νέων υπηρεσιών (microservices).
5. Ιχνηλασιμότητα και Έλεγχος (Audit Trail & Logging):
   Το σύστημα υιοθετεί πολιτική πλήρους καταγραφής (Logging). Κάθε ενέργεια που επιφέρει αλλαγή στα δεδομένα (π.χ. POST/PUT requests για πληρωμές ή αλλαγή καλύψεων) καταγράφεται σε ειδικό πίνακα στη βάση δεδομένων. Η καταγραφή περιλαμβάνει το “Timestamp”, το “UserID” και το αποτέλεσμα της εντολής, διασφαλίζοντας τη δυνατότητα αναδρομικού ελέγχου (Non-repudiation).

5. Πραγμάτωση Συστήματος

5.1. Περιβάλλον Ανάπτυξης και Εργαλεία

Για την υλοποίηση του Πληροφοριακού Συστήματος επιλέχθηκε η πλατφόρμα της Java, η οποία προσφέρει στιβαρότητα, ασφάλεια και δυνατότητα κλιμάκωσης (scalability) για τραπεζικές και ασφαλιστικές εφαρμογές.
Συγκεκριμένα χρησιμοποιήθηκαν:

- **Γλώσσα Προγραμματισμού, Java 17**. Επιλέχθηκε η έκδοση Long-Term Support (LTS) για τη διασφάλιση σταθερότητας και σύγχρονων χαρακτηριστικών απόδοσης.
- **Framework Ανάπτυξης, Spring Boot.** Επιλέχθηκε για τη δημιουργία RESTful Web Services, καθώς απλοποιεί τη ρύθμιση της εφαρμογής και ακολουθεί την αρχιτεκτονική μικρο-υπηρεσιών (Microservices/SOA).
- **Σύστημα Διαχείρισης, Apache Maven** (για τη διαχείριση εξαρτήσεων/βιβλιοθηκών).
- **Βάση Δεδομένων, MySQL** Επιλέχθηκε ως το κύριο σχεσιακό σύστημα διαχείρισης για την αποθήκευση των συμβολαίων και των πληρωμών. Για το περιβάλλον ανάπτυξης και τις δοκιμές χρησιμοποιήθηκε η in-memory βάση H2.
- **Frontend, HTML5 & CSS3 & Thymeleaf** (ως template engine της Java)

5.2. Υλοποίηση Βάσης Δεδομένων (Database Schema)

Ο σχεδιασμός της βάσης δεδομένων ακολουθεί το σχεσιακό μοντέλο. Ακολουθεί ο κώδικας SQL για τη δημιουργία των βασικών πινάκων.

**Σημείωση Αρχιτεκτονικής: Σε πραγματικό περιβάλλον παραγωγής, το Web Portal θα αντλούσε τα δεδομένα (Συμβόλαια, Πελάτες) μέσω API από το κεντρικό Core Insurance System (Legacy System) της εταιρείας. Για τις ανάγκες της παρούσας εργασίας και την αυτοτέλεια του πρωτοτύπου, υλοποιούμε ένα τοπικό σχήμα βάσης δεδομένων (Relational Schema) που προσομοιώνει την κεντρική βάση της εταιρείας.*

```
-- Πίνακας Πελατών

CREATE TABLE Customer (
 cust_id BIGINT AUTO_INCREMENT PRIMARY KEY,
 username VARCHAR(50) NOT NULL UNIQUE,
 full_name VARCHAR(100),
 afm VARCHAR(20)
);

-- Πίνακας Συμβολαίων

CREATE TABLE Policy (
 policy_id BIGINT AUTO_INCREMENT PRIMARY KEY,
 cust_id BIGINT,
 category VARCHAR(20),
 base_premium DECIMAL(10, 2),
 FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
);
-- Πίνακας Καλύψεων

CREATE TABLE Coverage (
 cov_id BIGINT AUTO_INCREMENT PRIMARY KEY,
 description VARCHAR(100),
 cost DECIMAL(10, 2)
);
```

```
-- Πίνακας Πληρωμών

CREATE TABLE Payment (
 pay_id BIGINT AUTO_INCREMENT PRIMARY KEY,
 policy_id BIGINT,
 amount DECIMAL(10, 2),
 payment_date DATE,
 status VARCHAR(20), -- 'COMPLETED', 'PENDING'
 FOREIGN KEY (policy_id) REFERENCES Policy(policy_id)
);


-- Πίνακας Καταγραφής Ενεργειών (Audit Log)
CREATE TABLE AuditLog (
 log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
 cust_id BIGINT,             -- Ποιος έκανε την ενέργεια
 action VARCHAR(100),        -- Τι ενέργεια έγινε (π.χ.
'UPDATE_COVERAGE')
 status VARCHAR(20),         -- Αποτέλεσμα (π.χ. 'SUCCESS', 'FAILED')
 ip_address VARCHAR(45),     -- Η IP του χρήστη για λόγους ασφαλείας
 timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
);
```

5.3. Υλοποίηση Λογικής Συστήματος (Backend Logic- Java)

Η λογική του συστήματος υλοποιήθηκε με χρήση Spring Boot Controllers και Services. Παρακάτω παρουσιάζεται η κλάση UnderwritingService, η οποία είναι υπεύθυνη για τον δυναμικό υπολογισμό του ασφαλίστρου όταν ο χρήστης προσθέτει ή αφαιρεί καλύψεις.

5.3.1. Κλάση Υπηρεσίας (Service Layer)

```
package com.insurance.portal.service;

import com.insurance.portal.model.Policy;
import com.insurance.portal.model.Coverage;
import com.insurance.portal.repository.PolicyRepository;
import com.insurance.portal.repository.CoverageRepository; import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.util.List;


@Service
public class UnderwritingService {

 @Autowired
 private PolicyRepository policyRepo;

 @Autowired
```

```
 private CoverageRepository coverageRepo;


 /**
 * Υπολογίζει το νέο ασφάλιστρο βάσει των επιλεγμένων καλύψεων.
* @param policyId Το ID του συμβολαίου
* @param coverageIds Λίστα με τα IDs των καλύψεων που επέλεξε ο
πελάτης
* @return Το συνολικό ποσό πληρωμής
 */


 public BigDecimal calculateNewPremium(Long policyId, List<Long>
coverageIds) {
 // 1. Ανάκτηση του συμβολαίου από τη Βάση
 Policy policy = policyRepo.findById(policyId)
 .orElseThrow(() -> new RuntimeException("Policy not found"));

 // 2. Αρχικοποίηση με το βασικό ασφάλιστρο
 BigDecimal totalPremium = policy.getBasePremium();

 // 3. Προσθήκη κόστους για κάθε επιλεγμένη κάλυψη
 List<Coverage> selectedCoverages =
coverageRepo.findAllById(coverageIds);

 for (Coverage cov : selectedCoverages) {
 totalPremium = totalPremium.add(cov.getCost());  }

 return totalPremium;
 }


 /**
 * Ενημερώνει το συμβόλαιο με το νέο ασφάλιστρο.  */

 public void updatePolicy(Long policyId, BigDecimal newPremium) {

 Policy policy = policyRepo.findById(policyId).get();

policy.setTotalPremium(newPremium);

policyRepo.save(policy);
 }
}
```

5.3.2. Κλάση Ελέγχου (REST Controller)

Αυτή η κλάση δέχεται τα αιτήματα από το Web Portal (Frontend).

```
package com.insurance.portal.controller;
```

```
import com.insurance.portal.service.UnderwritingService; import
org.springframework.beans.factory.annotation.Autowired; import
org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/policy")
public class PolicyController {

 @Autowired
 private UnderwritingService underwritingService;

 // Endpoint: POST /api/policy/calculate
 @PostMapping("/calculate")
 public Map<String, Object> calculatePremium(@RequestBody Map<String,
Object> payload) {

 // Λήψη δεδομένων από το JSON αίτημα
 Long policyId = Long.valueOf(payload.get("policyId").toString());

  List<Long> coverageIds = (List<Long>) payload.get("coverageIds");

 // Κλήση της υπηρεσίας υπολογισμού
 BigDecimal newPremium =
underwritingService.calculateNewPremium(policyId, coverageIds);

 // Επιστροφή αποτελέσματος σε JSON
 return Map.of(
 "status", "success",
 "newPremium", newPremium,
 "message", "Ο υπολογισμός ολοκληρώθηκε επιτυχώς."  );
 }
}
```

5.3.3. Υλοποίηση Ασφάλειας και Εξουσιοδότησης (Spring Security)

Η ασφάλεια του συστήματος βασίζεται στο πλαίσιο Spring Security, το οποίο αναλαμβάνει την πιστοποίηση (Authentication) και την εξουσιοδότηση (Authorization). Η υλοποίηση διασφαλίζει ότι κάθε αίτημα HTTP ελέγχεται πριν φτάσει στον Controller.

Συγκεκριμένα, εφαρμόζονται τα εξής μέτρα:

1. Κρυπτογράφηση Κωδικών: Οι κωδικοί πρόσβασης δεν αποθηκεύονται ποτέ ως απλό κείμενο, αλλά κρυπτογραφούνται με τον αλγόριθμο BCrypt κατά την εγγραφή.
2. Έλεγχος Ρόλων (RBAC): Χρησιμοποιείται ο μηχανισμός SecurityFilterChain για να οριστεί ποια URL είναι προσβάσιμα από ποιους ρόλους.
3. Stateful Authentication: Για τις ανάγκες του Portal, χρησιμοποιείται Session-based authentication.

Ακολουθεί η κλάση διαμόρφωσης ασφάλειας (SecurityConfig.java):

```
package com.insurance.portal.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecur
ity;
import
org.springframework.security.config.annotation.web.configuration.Enab
leWebS ecurity;
import org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws  Exception {
 http
 .csrf().disable() // Απενεργοποίηση για απλότητα στο demo

.authorizeHttpRequests((requests) -> requests


 // Δημόσια προσβάσιμες σελίδες (Login, Register, CSS/JS)
.requestMatchers("/", "/login", "/register", "/css/**",
"/js/**").permitAll()

// Σελίδες που απαιτούν ρόλο Πελάτη
 .requestMatchers("/dashboard/**",
"/api/policy/**").hasRole("CUSTOMER")

// Σελίδες διαχειριστών (για μελλοντική χρήση)
 .requestMatchers("/admin/**").hasRole("ADMIN")

// Όλα τα άλλα απαιτούν απλή σύνδεση
 .anyRequest().authenticated()
 )

 .formLogin((form) -> form
 .loginPage("/login")
 .defaultSuccessUrl("/dashboard", true)
 .permitAll()
 )

 .logout((logout) -> logout.permitAll());

 return http.build();
 }
```

```
 @Bean
 public PasswordEncoder passwordEncoder() {

 // Χρήση αλγορίθμου BCrypt για ασφαλή hashing κωδικών
return new BCryptPasswordEncoder();
 }
}
```

5.3.4 Διαχείριση Εξαιρέσεων (Global Exception Handling)

Για τη διασφάλιση της ομαλής λειτουργίας του συστήματος, υλοποιήθηκε μηχανισμός καθολικής διαχείρισης σφαλμάτων μέσω της Annotation @ControllerAdvice. Αυτό επιτρέπει στο σύστημα να εμφανίζει φιλικά μηνύματα προς τον χρήστη αντί για τεχνικά μηνύματα λάθους (Stack Traces), όταν προκύπτουν εξαιρέσεις.

```
package com.insurance.portal.exception;

import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.servlet.ModelAndView;

@ControllerAdvice
public class GlobalExceptionHandler {

 // Χειρισμός περίπτωσης που δεν βρέθηκε εγγραφή (π.χ. λάθος ID
συμβολαίου)  @ExceptionHandler(ResourceNotFoundException.class)
 public ModelAndView handleNotFound(ResourceNotFoundException ex) {

       ModelAndView mav = new ModelAndView("error-page");

       mav.addObject("errorMessage", "Η εγγραφή που αναζητάτε δεν
βρέθηκε.");

       return mav;
 }
 // Χειρισμός γενικών τεχνικών σφαλμάτων (π.χ. Database connection
error)  @ExceptionHandler(Exception.class)
 public ModelAndView handleGeneralError(Exception ex) {

       ModelAndView mav = new ModelAndView("error-page");

       mav.addObject("errorMessage", "Προέκυψε ένα απροσδόκητο
σφάλμα.  Παρακαλώ δοκιμάστε αργότερα.");

       return mav;
 }
}
```

5.3.5 Έλεγχος Λογισμικού (Unit Testing)

Για τη διασφάλιση της ορθής λειτουργίας της επιχειρησιακής λογικής (Business Logic) και την αποφυγή σφαλμάτων (bugs) κατά τη διάρκεια της ανάπτυξης, υλοποιήθηκαν αυτοματοποιημένοι έλεγχοι μονάδας (Unit Tests). Συγκεκριμένα, χρησιμοποιήθηκε το πλαίσιο JUnit 5 σε συνδυασμό με το Spring Boot Test. Σκοπός ήταν να επιβεβαιωθεί ότι η κρίσιμη υπηρεσία υπολογισμού ασφαλίστρου (UnderwritingService) λειτουργεί σωστά και παράγει τα αναμενόμενα οικονομικά αποτελέσματα.
Ακολουθεί ο κώδικας της κλάσης ελέγχου UnderwritingServiceTest.java:

```
package com.insurance.portal.service;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import java.math.BigDecimal;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertEquals;

@SpringBootTest
class UnderwritingServiceTest {

    @Autowired
    private UnderwritingService underwritingService;

    @Test
    void testCalculateNewPremium() {
        // Σενάριο Δοκιμής:
        // Έστω ένα συμβόλαιο με ID=1 (Βασικό Ασφάλιστρο = 100.00€)
        // Ο χρήστης επιλέγει την κάλυψη με ID=2 (Οδική Βοήθεια =
30.00€)

        Long policyId = 1L;
        List<Long> selectedCoverageIds = List.of(2L);

        // Εκτέλεση της μεθόδου υπολογισμού
        BigDecimal result =
underwritingService.calculateNewPremium(policyId,
selectedCoverageIds);

        // Έλεγχος (Assertion): Το αποτέλεσμα πρέπει να είναι ακριβώς
130.00€
        assertEquals(new BigDecimal("130.00"), result, "Ο υπολογισμός
του ασφαλίστρου δεν είναι σωστός.");
    }
}
```

Για την επίτευξη λοιπόν, υψηλών προδιαγραφών ποιότητας και την εξασφάλιση της εύρυθμης λειτουργίας του Πληροφοριακού Συστήματος, υιοθετήθηκε μια διττή προσέγγιση που συνδυάζει την πρόληψη σφαλμάτων (5.3.4) κατά την ανάπτυξη και την αντιμετώπιση σφαλμάτων (5.3.5) κατά την εκτέλεση. Οι δύο αυτές διαδικασίες, αν και διακριτές, λειτουργούν συμπληρωματικά για τη θωράκιση της εφαρμογής.

1. Έλεγχος Μονάδας (Unit Testing) - Ο Μηχανισμός Πρόληψης

Ο Έλεγχος Μονάδας (ενότητα 5.3.5) εστιάζει στην επαλήθευση της ορθότητας του κώδικα πριν αυτός τεθεί σε παραγωγική λειτουργία.

Σκοπός του είναι να πιστοποιήσει ότι η επιχειρησιακή λογική (Business Logic) παράγει μαθηματικά και λογικά ακριβή αποτελέσματα. Συγκεκριμένα, μέσω του UnderwritingServiceTest, επιβεβαιώνεται ότι ο αλγόριθμος υπολογισμού ασφαλίστρου λειτουργεί σωστά για κάθε πιθανό σενάριο συνδυασμού καλύψεων.

Έτσι ελαχιστοποιείται ο κίνδυνος εισαγωγής λογικών σφαλμάτων (bugs) στο σύστημα και διασφαλίζεται ότι μελλοντικές αλλαγές στον κώδικα δεν θα αλλοιώσουν την υφιστάμενη λειτουργικότητα (Regression Testing). Η διαδικασία αυτή αυτοματοποιείται μέσω του CI pipeline, λειτουργώντας ως η πρώτη γραμμή άμυνας της ποιότητας του λογισμικού.

2. Καθολική Διαχείριση Εξαιρέσεων (Global Exception Handling) - Ο Μηχανισμός Αντιμετώπισης

Η Διαχείριση Εξαιρέσεων (ενότητα 5.3.4) εστιάζει στη σταθερότητα του συστήματος κατά τη διάρκεια της χρήσης του από τους τελικούς χρήστες.

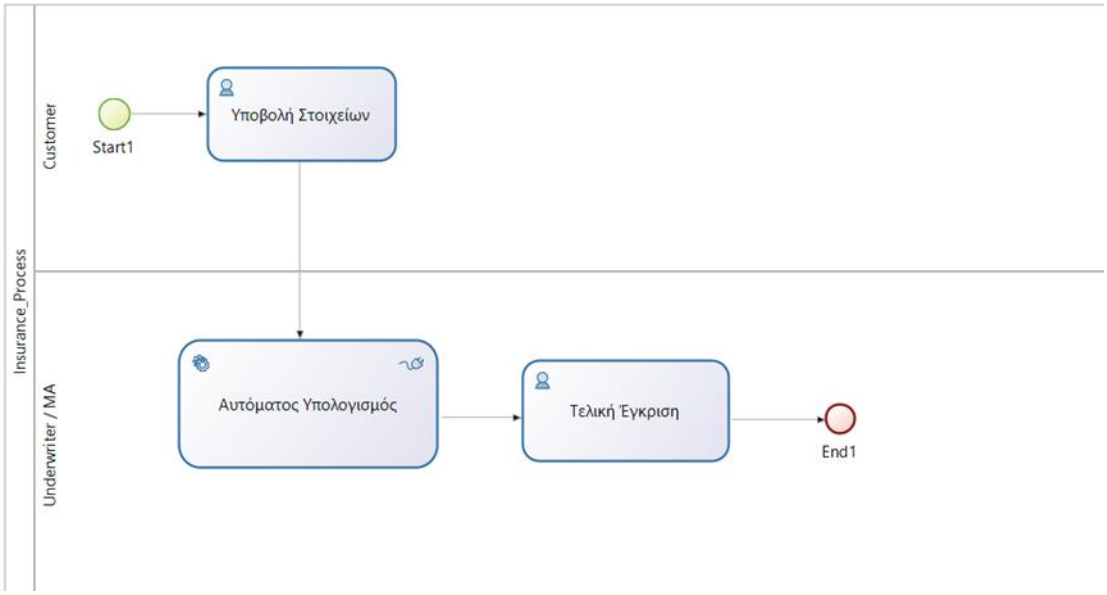
Σκοπός είναι η διαχείριση απρόβλεπτων καταστάσεων (runtime errors), όπως η απώλεια σύνδεσης με τη βάση δεδομένων ή η αναζήτηση μη υπαρκτών πόρων, με τρόπο ελεγχόμενο και φιλικό προς τον χρήστη. Μέσω της κλάσης GlobalExceptionHandler και του annotation @ControllerAdvice, το σύστημα «παγιδεύει» τα τεχνικά σφάλματα.

Έτσι αποτρέπεται η κατάρρευση (crash) της εφαρμογής και η εμφάνιση ακατανόητων τεχνικών μηνυμάτων (stack traces) στον χρήστη. Αντ' αυτού, το σύστημα ανακάμπτει ομαλά (graceful degradation), ενημερώνοντας τον χρήστη με σαφήνεια για το πρόβλημα, διατηρώντας έτσι υψηλό επίπεδο εμπειρίας χρήστη (User Experience) και επαγγελματισμού.

Συνοψίζοντας, ο συνδυασμός του Ελέγχου Μονάδας και της Καθολικής Διαχείρισης Εξαιρέσεων διασφαλίζει την ακεραιότητα του Πληροφοριακού Συστήματος σε δύο επίπεδα. Ο μεν πρώτος επαληθεύει την ορθότητα της επιχειρησιακής λογικής (Business Logic) κατά το στάδιο της ανάπτυξης, ενώ η δεύτερη εγγυάται τη σταθερότητα και την ομαλή ανάκαμψη (Robustness) της εφαρμογής σε περιβάλλον παραγωγής.

5.4. Ενορχήστρωση Επιχειρησιακών Διεργασιών με Bonita BPM

Η χρήση του Bonita BPM διασφαλίζει τον διαχωρισμό της επιχειρησιακής λογικής από την τεχνική υλοποίηση. Η δραστηριότητα «Αυτόματος Υπολογισμός»υλοποιήθηκε ως Service Task, το οποίο μέσω REST POST Connector καλεί το endpoint http://localhost:8080/api/policies/calculate του Spring Boot. Η επικοινωνία πραγματοποιείται με ανταλλαγή δεδομένων JSON, όπου ορίζεται η επικεφαλίδα Content-Type: application/json για συμβατότητα με τον Controller της Java. Σημειώνεται ότι η «Τελική Έγκριση» στο Bonita αφορά μόνο σύνθετες περιπτώσεις (εξαιρέσεις).



Εικόνα 10: Μοντελοποίηση της επιχειρησιακής διαδικασίας «Insurance Process» σε περιβάλλον Bonita BPM Studio.

Εικόνα 11: Παραμετροποίηση του REST POST Connector για την κλήση του εξωτερικού API υπολογισμού ασφαλίστρου.

5.5. Αυτοματοποίηση και Containerization (DevOps Approach)

Στο πλαίσιο της σύγχρονης ανάπτυξης λογισμικού και σύμφωνα με τις απαιτήσεις για ευελιξία, υιοθετήθηκε μια προσέγγιση DevOps. Σκοπός είναι η γεφύρωση του χάσματος μεταξύ ανάπτυξης (Development) και λειτουργίας (Operations), επιτρέποντας τη γρήγορη και ασφαλή διάθεση νέων εκδόσεων.

Για την επίτευξη αυτού, η εφαρμογή «πακεταρίστηκε» (containerized) με χρήση της τεχνολογίας Docker. Αυτό διασφαλίζει ότι το σύστημα μπορεί να τρέξει με τον ίδιο ακριβώς τρόπο σε οποιοδήποτε περιβάλλον, χωρίς προβλήματα εξαρτήσεων.

Ακολουθεί το αρχείο Dockerfile που δημιουργήθηκε για την παραγωγή του ειδώλου (image) της εφαρμογής:

```
# Χρήση επίσημου OpenJDK image (Java 17)
FROM openjdk:17-jdk-alpine

# Ορισμός μεταβλητής για το αρχείο .jar της εφαρμογής ARG
JAR_FILE=target/insurance-portal-1.0.0.jar

# Αντιγραφή του jar στο container
COPY ${JAR_FILE} app.jar

# Άνοιγμα της θύρας 8080
```

```
EXPOSE 8080

# Εντολή εκκίνησης της εφαρμογής
ENTRYPOINT ["java","-jar","/app.jar"]
```

Επιπλέον, για τη διαδικασία Continuous Integration (CI), σχεδιάστηκε ένα βασικό pipeline το οποίο εκτελεί αυτόματα τα Unit Tests κάθε φορά που γίνεται νέο commit στον κώδικα, διασφαλίζοντας την ποιότητα του λογισμικού πριν την παραγωγή.

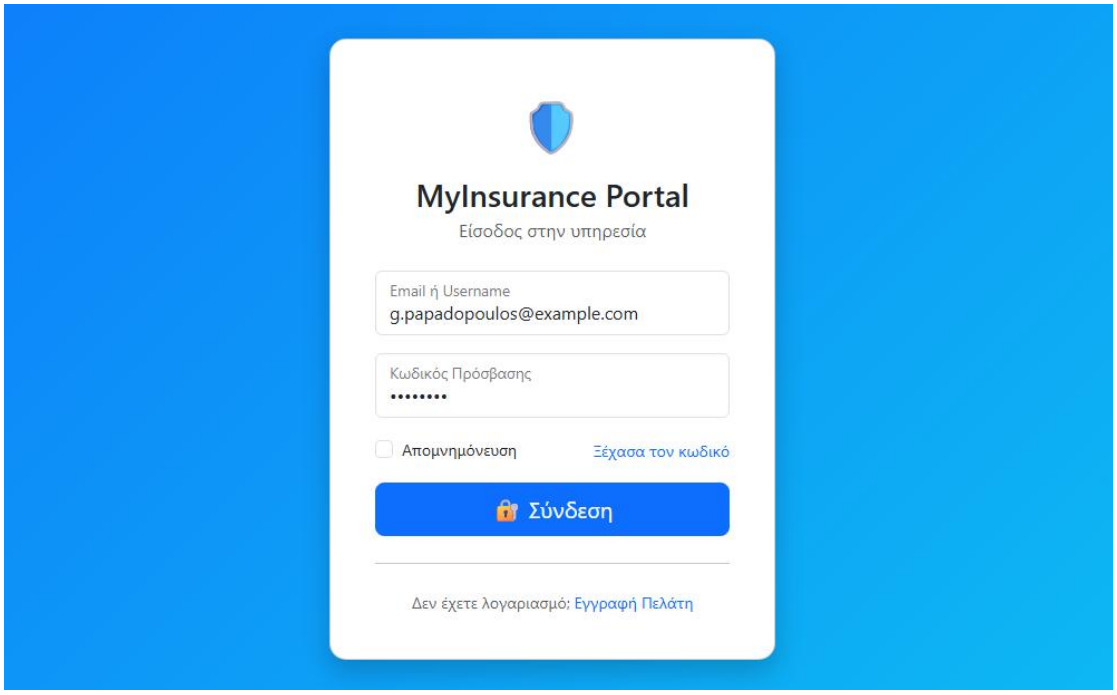
5.6. Διεπαφή Χρήστη (User Interface)

Η σχεδίαση της διεπαφής χρήστη (UI) βασίστηκε στις αρχές της ευχρηστίας (usability) και της απλότητας. Το σύστημα αναπτύχθηκε ως διαδικτυακή εφαρμογή (Web Application) χρησιμοποιώντας τεχνολογίες HTML5, CSS3 και το πλαίσιο Bootstrap, εξασφαλίζοντας την ορθή εμφάνιση σε όλες τις συσκευές (Responsive Design).
Παρακάτω παρουσιάζονται οι κύριες οθόνες του συστήματος.

5.6.1. Οθόνη Ταυτοποίησης (Login Screen)

Η είσοδος στο σύστημα αποτελεί το πρώτο σημείο επαφής του χρήστη με την εφαρμογή.

Η οθόνη εισόδου υλοποιεί το πρωτόκολλο ασφαλείας που ορίστηκε στο Κεφάλαιο



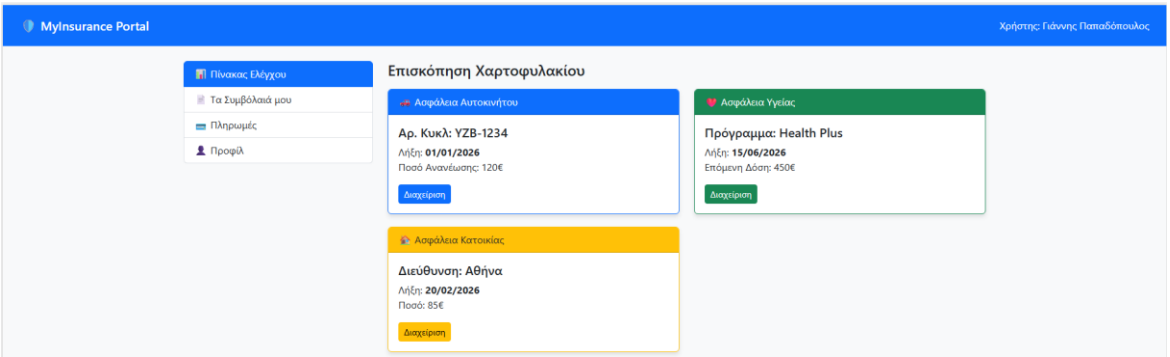
Εικόνα 12: Οθόνη εισόδου στο MyInsurance Portal

5.3.3. Ο χρήστης καλείται να εισάγει τα διαπιστευτήριά του (Email και Κωδικό). Το σύστημα ελέγχει τα στοιχεία μέσω του Spring Security και, εφόσον είναι έγκυρα, δημιουργεί ένα κρυπτογραφημένο Session και ανακατευθύνει τον χρήστη στον Πίνακα Ελέγχου. Σε περίπτωση λάθους, εμφανίζεται κατάλληλο μήνυμα σφάλματος, ενώ παρέχεται και η δυνατότητα ανάκτησης κωδικού («Ξέχασα τον κωδικό»).

5.6.2. Πίνακας Ελέγχου (Dashboard)

Μετά την επιτυχή ταυτοποίηση, ο χρήστης μεταφέρεται στον κεντρικό Πίνακα Ελέγχου. Σε αυτή την οθόνη παρέχεται μια συγκεντρωτική εικόνα του ασφαλιστικού χαρτοφυλακίου.

Όπως φαίνεται στην εικόνα, τα συμβόλαια ομαδοποιούνται σε κάρτες (Cards) ανά κλάδο (Αυτοκίνητο, Υγεία, Κατοικία, Ζωή). Κάθε κάρτα παρουσιάζει κρίσιμες πληροφορίες όπως τον αριθμό συμβολαίου, την ημερομηνία λήξης και το καθεστώς πληρωμής, ενώ παρέχει άμεση πρόσβαση στη λειτουργία διαχείρισης.

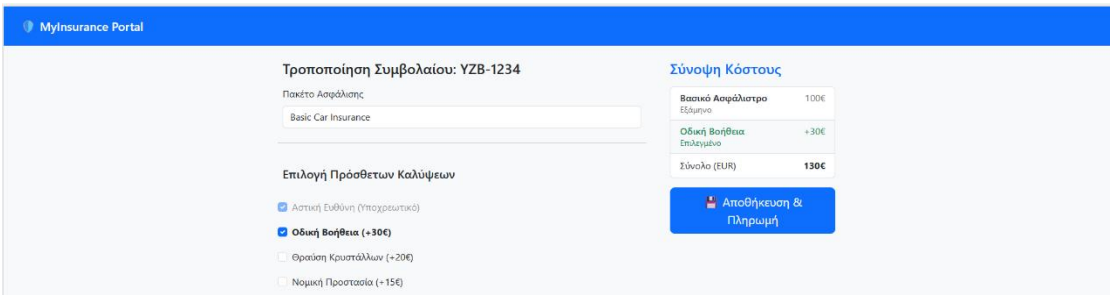


Εικόνα 13: Κεντρικός Πίνακας Ελέγχου χρήστη.

5.6.3. Διαχείριση και Τροποποίηση Καλύψεων (Underwriting)

Επιλέγοντας «Διαχείριση» σε ένα συμβόλαιο, ο χρήστης μεταβαίνει στη σελίδα λεπτομερειών. Εδώ υλοποιείται η δυναμική λειτουργία του Underwriting που περιγράφηκε στον κώδικα (Κεφάλαιο 5.3).

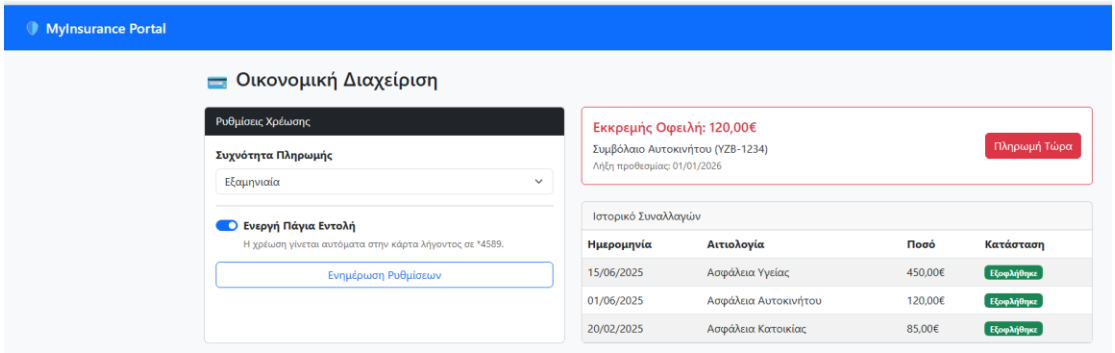
Ο χρήστης έχει τη δυνατότητα να επιλέξει ή να αποεπιλέξει πρόσθετες καλύψεις (π.χ. Οδική Βοήθεια, Θραύση Κρυστάλλων) μέσω των αντίστοιχων checkboxes. Το σύστημα, επικοινωνώντας με το Backend (Java REST API), υπολογίζει σε πραγματικό χρόνο το νέο ασφάλιστρο και το εμφανίζει στο δεξί μέρος της οθόνης («Σύνολο Πληρωμής»), πριν ο χρήστης προχωρήσει στην οριστική αποθήκευση.



Εικόνα 14: Δυναμική τροποποίηση καλύψεων και υπολογισμός κόστους.

5.6.4. Οικονομική Διαχείριση και Πληρωμές

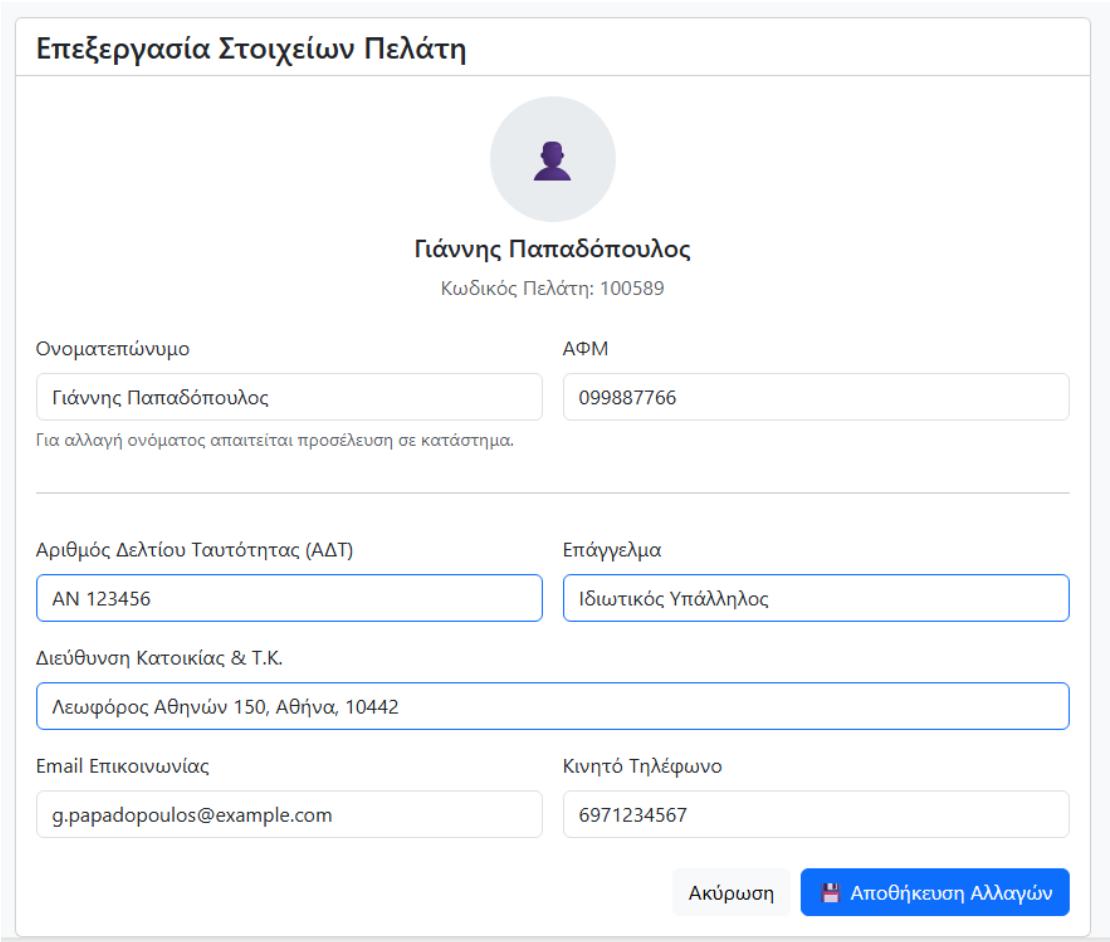
Αυτή η οθόνη δείχνει το ιστορικό πληρωμών, επιτρέπει την εξόφληση και τη ρύθμιση της πάγιας εντολής.



Εικόνα 15: Οθόνη οικονομικής διαχείρισης και ιστορικού πληρωμών.

5.6.5. Επεξεργασία Στοιχείων Πελάτη

Αυτή η οθόνη δείχνει τη φόρμα επεξεργασίας προσωπικών στοιχείων.



Εικόνα 16: Φόρμα επεξεργασίας προσωπικών στοιχείων πελάτη.

5.7. Τεκμηρίωση Διεπαφών (API Documentation)

Καθώς το σύστημα βασίζεται σε αρχιτεκτονική RESTful Services, η τεκμηρίωση των endpoints είναι κρίσιμη για τη διασυνδεσιμότητα με άλλα συστήματα (π.χ. Τράπεζες ή mobile apps).

Για τον σκοπό αυτό ενσωματώθηκε η βιβλιοθήκη SpringDoc OpenAPI (Swagger UI). Αυτό επιτρέπει την αυτόματη παραγωγή τεκμηρίωσης και την εκτέλεση δοκιμαστικών κλήσεων.

Παράδειγμα JSON απόκρισης από το endpoint /api/policy/calculate:

```
{
 "status": "success",
 "policyId": 1234,
 "oldPremium": 100.00,
 "addedCoveragesCost": 30.00,
 "newTotalPremium": 130.00,
 "message": "Ο υπολογισμός ολοκληρώθηκε επιτυχώς."
}
```

5.8. Πίνακας Ιχνηλασιμότητας Απαιτήσεων (Traceability Matrix)

Για την πιστοποίηση της πληρότητας του συστήματος και τη διασφάλιση ότι όλες οι λειτουργικές απαιτήσεις που ορίστηκαν στο Product Backlog έχουν υλοποιηθεί και ελεγχθεί επιτυχώς, συντάχθηκε ο Πίνακας Ιχνηλασιμότητας (RTM). Ο πίνακας αυτός χαρτογραφεί τη συσχέτιση μεταξύ των Ιστοριών Χρήστη (User Stories), των δομικών στοιχείων του κώδικα (Classes) και των σεναρίων ελέγχου.

ID Απαίτησης	Περιγραφή (User Story)	Υλοποίηση (Component/Class)	Έλεγχος (Test Case/Verification)	Κατάσταση
US-01	Προβολή Χαρτοφυλακίου ανά κλάδο	DashboardController, PolicyRepository	UI Test / Manual Verification	Ολοκληρώθηκε
US-02	Τροποποίηση Προφίλ Πελάτη	CustomerService, Customer Entity	updateProfile() Unit Test	Ολοκληρώθηκε
US-03	Προβολή ημερομηνιών ανανέωσης	DashboardController, PolicyRepository	UI Test / Manual Verification	Ολοκληρώθηκε
US-04	Διαχείριση Πληρωμών	PaymentController, Payment Entity	Integration Test (Mock Bank)	Ολοκληρώθηκε
US-05	Ηλεκτρονική Πληρωμή (Online)	PaymentGateway Service	Transaction Log Check	Ολοκληρώθηκε
US-06	Τροποποίηση Καλύψεων (Underwriting)	UnderwritingService	UnderwritingServiceTest	Ολοκληρώθηκε
SEC-01	Ασφάλεια & Ταυτοποίηση	SecurityConfig, BCryptEncoder	Login Security Check	Ολοκληρώθηκε

Πίνακας 7: Πίνακας Ιχνηλασιμότητας Απαιτήσεων

6. Συμπεράσματα και Μελλοντικές Επεκτάσεις

6.1. Συμπεράσματα

Στην παρούσα εργασία ολοκληρώθηκε η μελέτη, ο σχεδιασμός και η ανάπτυξη ενός Πληροφοριακού Συστήματος (Web Portal) για μια ασφαλιστική εταιρεία. Μέσα από την εφαρμογή της Μεθοδολογίας Εύπλαστων Συστημάτων (SSM), αναδείχθηκε η ανάγκη μετάβασης από τις παραδοσιακές, χρονοβόρες διαδικασίες σε ένα μοντέλο ψηφιακής αυτοεξυπηρέτησης. Η επιτυχής πραγμάτωση του συστήματος βασίστηκε στον συνδυασμό της γλώσσας Java (Spring Boot) για την επιχειρησιακή λογική και της πλατφόρμας Bonita BPM για την ενορχήστρωση των διεργασιών. Η προσέγγιση αυτή επέτρεψε τη μετατροπή των θεωρητικών μοντέλων της SSM σε ένα εκτελέσιμο και ευέλικτο πληροφοριακό σύστημα

Η υιοθέτηση της Agile μεθοδολογίας (SCRUM) επέτρεψε την ευέλικτη ανάπτυξη του λογισμικού, εστιάζοντας στις λειτουργίες που προσφέρουν τη μεγαλύτερη αξία στον πελάτη, όπως η άμεση προβολή συμβολαίων και η δυναμική τροποποίηση καλύψεων (Underwriting). Η τεχνική υλοποίηση με χρήση Java (Spring Boot) και σύγχρονων εργαλείων διεπαφής απέδειξε ότι είναι εφικτή η δημιουργία ενός ασφαλούς και φιλικού περιβάλλοντος για τους ασφαλισμένους με έμφαση στη διασφάλιση της ποιότητας του λογισμικού μέσω μιας ολιστικής προσέγγισης ελέγχου: υιοθέτηση αυτοματοποιημένων ελέγχων (Unit Testing) κατά τη φάση της ανάπτυξης για την ορθότητα των κρίσιμων οικονομικών υπολογισμών (πρόληψη) και ενσωμάτωση μηχανισμών καθολικής διαχείρισης σφαλμάτων (Exception Handling) για την θωράκιση του συστήματος έναντι απρόβλεπτων καταστάσεων λειτουργίας (αντιμετώπιση).

Μέσω του Bonita BPM, επιτεύχθηκε ο πλήρης διαχωρισμός των επιχειρησιακών κανόνων από τον τεχνικό κώδικα. Αυτό σημαίνει ότι μελλοντικές αλλαγές στη ροή έγκρισης ενός συμβολαίου μπορούν να υλοποιηθούν άμεσα στο περιβάλλον του Bonita χωρίς την ανάγκη εκτεταμένων τροποποιήσεων στον πηγαίο κώδικα της εφαρμογής

Τα βασικά οφέλη που προκύπτουν από τη λειτουργία του συστήματος είναι:

- Εξοικονόμηση Πόρων: Μείωση του φόρτου εργασίας στο τηλεφωνικό κέντρο και το τμήμα διαχείρισης συμβολαίων.
- Βελτίωση Εμπειρίας Πελάτη: Δυνατότητα 24ωρης εξυπηρέτησης και άμεσης διεκπεραίωσης αιτημάτων.
- Μείωση Λαθών: Η αυτοματοποίηση των υπολογισμών και των πληρωμών ελαχιστοποιεί το ανθρώπινο λάθος.

Τέλος, η υιοθέτηση μιας DevOps φιλοσοφίας, με τη χρήση REST Connectors για τη διασύνδεση των εργαλείων και την containerized αρχιτεκτονική (Docker), διασφαλίζει ότι η ασφαλιστική πύλη είναι έτοιμη για συνεχή διάθεση (Continuous Delivery) και κλιμάκωση ανάλογα με τις ανάγκες της αγορά

6.2. Μελλοντικές Επεκτάσεις

Το σύστημα σχεδιάστηκε με ανοικτή αρχιτεκτονική (Service Oriented Architecture), επιτρέποντας την εύκολη προσθήκη νέων λειτουργιών στο μέλλον. Προτείνονται οι εξής επεκτάσεις:

1. Ανάπτυξη Mobile Application: Δημιουργία εγγενούς εφαρμογής (Native App) για iOS και Android, ώστε να αξιοποιούνται λειτουργίες της συσκευής (π.χ. βιομετρική είσοδος με FaceID, ειδοποιήσεις Push Notifications για ανανεώσεις).
2. Ενσωμάτωση Τεχνητής Νοημοσύνης (AI Chatbot): Προσθήκη ψηφιακού βοηθού που θα απαντά αυτόματα σε ερωτήσεις και θα βοηθά στη συμπλήρωση δηλώσεων ατυχήματος.
3. Διασύνδεση με Gov.gr: Αυτόματη άντληση στοιχείων (π.χ. στοιχεία οχήματος, ΚΤΕΟ) μέσω των κρατικών API για την ταχύτερη έκδοση προσφορών.
4. Telematics: Διασύνδεση με συσκευές τηλεμετρίας στο αυτοκίνητο για την παροχή εκπτώσεων βάσει οδηγικής συμπεριφοράς (Usage-Based Insurance).
5. Αυτοματοποιημένη Εκκαθάριση Ζημιών (Straight-Through Processing - STP): Υλοποίηση μηχανισμού άμεσης έγκρισης αποζημιώσεων για μικρο-ζημιές χωρίς ανθρώπινη παρέμβαση. Μέσω ενός «Μηχανισμού Επιχειρησιακών Κανόνων» (Business Rules Engine), το σύστημα θα αξιολογεί αυτόματα το αίτημα βάσει κριτηρίων (π.χ. ιστορικό πελάτη, ύψος ζημιάς < 300€, φωτογραφική τεκμηρίωση AI) και θα εκτελεί την πληρωμή σε δευτερόλεπτα. Αυτό μειώνει δραστικά το διοικητικό κόστος και εκτοξεύει την ικανοποίηση του πελάτη.

7. Βιβλιογραφία

Bonitasoft. (2025). *Bonita BPM documentation and connectors guide*. https://documentation.bonitasoft.com/

Checkland, P. (1981). *Systems thinking, systems practice*. John Wiley & Sons.

Checkland, P., & Scholes, J. (1990). *Soft systems methodology in action*. John Wiley & Sons.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Walls, C. (2022). *Spring Boot in action*. Manning Publications.

Βασιλακόπουλος, Γ. (2025). *Σχεδιασμός Βάσεων Δεδομένων*. Τσότρας.

Βασιλακόπουλος, Γ. (2026). *Πληροφοριακά συστήματα: Προσεγγίσεις, Ανάπτυξη, Πραγμάτωση*. Τσότρας.